

第 5 章 数据库系统

在当今的知识经济时代，信息是经济发展的战略资源，信息技术已经成为社会生产力中重要的组成部分。人们充分认识到，数据库是信息化社会中信息资源管理与开发利用的基础，当今的计算机信息系统也都以数据库技术为基础。对于一个国家来说，数据库的建设规模和使用水平已成为衡量该国信息化程度的重要标志。因此，数据库课程是计算机领域中的一门重要课程，也是系统分析师必须要掌握的专业知识与技能。

5.1 数据库管理系统

数据库是长期存储在计算机内的、有组织的、可共享的数据集合，数据库系统是指在计算机信息系统中引入数据库后的系统，一般由数据库、数据库管理系统（DataBase Management System, DBMS）、应用系统、数据库管理员（DataBase Administrator, DBA）和用户构成。

5.1.1 概述

数据库管理系统是一种操纵和管理数据库的大型软件，用于建立、使用和维护数据库。它对数据库进行统一的管理和控制，以保证数据库的安全性和完整性。用户通过 DBMS 访问数据库中的数据，数据库管理员也通过 DBMS 进行数据库的维护工作。DBMS 可以支持多个应用程序和用户用不同的方法在同时或不同时刻去建立、修改和询问数据库。大部分 DBMS 提供数据定义语言（Data Definition Language, DDL）和数据操作语言（Data Manipulation Language, DML），供用户定义数据库的模式结构与权限约束，实现对数据的追加、删除等操作。

简单地讲，数据库管理系统是一个能够提供数据录入、修改、查询的数据操作软件，具有数据定义、数据操作、数据存储与管理、数据维护、通信等功能，且能够允许许多用户使用。

5.1.2 三级划分法

数据库系统的结构划分可以有多种不同的层次或不同的角度，典型的是三级划分法，就是把数据库系统划分为三级模式，在三级模式之间提供两级映射，这两级映射保证了数据库中的数据具有较高的物理独立性和逻辑独立性。

1. 三级模式

数据库系统的三级模式如图 5-1 所示，从图 5-1 中可以看出，数据库系统由外模式、概念模式和内模式三级构成。

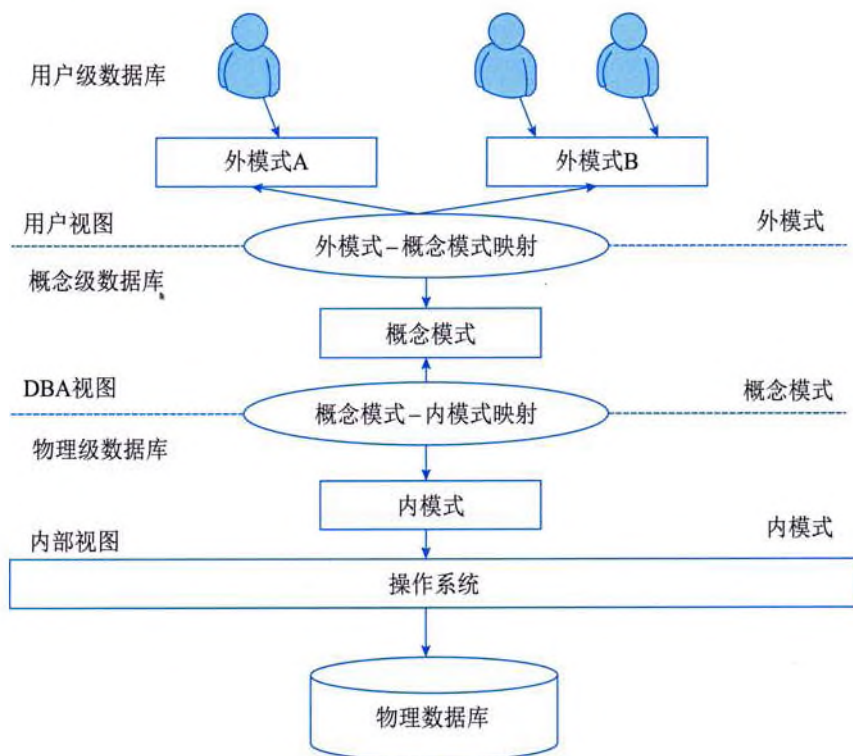


图 5-1 数据库系统结构层次图

外模式也称为子模式或用户模式，对应于用户级数据库。外模式用以描述用户（包括程序员和终端用户）看到或使用的那部分数据的逻辑结构，是数据库用户的数据视图，是与某一应用有关的数据的逻辑表示。用户根据外模式用数据操作语句或应用程序去操作数据库中的数据。外模式主要描述组成用户视图的各个记录的组成、相互关系、数据项的特征、数据的安全性和完整性约束条件。一个数据库可以有多个外模式，一个应用程序只能使用一个外模式。

概念模式也称为模式或逻辑模式，对应于概念级数据库。概念模式是数据库中全体数据的逻辑结构和特征的描述，是所有用户的公共数据视图，用以描述现实世界中的实体及其性质与联系，定义记录、数据项、数据的完整性约束条件及记录之间的联系。概念模式通常还包含访问控制、保密定义和完整性检查等方面的内容，以及概念/物理之间的映射。一个数据库只有一个概念模式。

内模式对应于物理级数据库，是数据物理结构和存储方式的描述，是数据在数据库内部的表示方式。内模式不同于物理层，它假设外存是一个无限的线性地址空间。内模式定义的是存储记录的类型、存储域的表示和存储记录的物理顺序，以及索引和存储路径等数据的存储组织。一个数据库只有一个内模式。

在数据库系统的三级模式中，模式是数据库的中心与关键；内模式依赖于模式，独立于外模式和存储设备；外模式面向具体的应用，独立于内模式和存储设备；应用程序依赖于外模式，

独立于模式和内模式。

2. 两级独立性

数据库系统的两级独立性是指物理独立性和逻辑独立性。三个抽象级别之间通过两级映射（外模式/模式映射和模式/内模式映射）进行相互转换，使得数据库的三级模式形成一个统一的整体。

物理独立性是指用户的应用程序与存储在磁盘上的数据库中的数据是相互独立的，当数据的物理存储改变时，应用程序不需要改变。物理独立性存在于概念模式和内模式之间的映射转换，说明物理组织发生变化时应用程序的独立程度。

逻辑独立性是指用户的应用程序与数据库中的逻辑结构是相互独立的，当数据的逻辑结构改变时，应用程序不需要改变。逻辑独立性存在于外模式和概念模式之间的映射转换，说明概念模式发生变化时应用程序的独立程度。相对来说，逻辑独立性比物理独立性更难实现。

5.1.3 数据模型

数据模型是现实世界数据特征的抽象。通过这种抽象，可以将现实世界的问题转化到计算机上进行分析与解决。数据模型所描述的内容包括3个部分，分别是数据结构、数据操作和数据约束。其中，数据结构主要描述数据的类型、内容、性质和数据间的联系等。数据结构是数据模型的基础，数据操作和数据约束都建立在数据结构上。不同的数据结构具有不同的操作和约束。数据操作主要描述在相应的数据结构上的操作类型和操作方式。数据约束主要描述数据结构内数据间的语法和词义联系、它们之间的制约和依存关系，以及数据动态变化的规则，以保证数据的正确、有效和相容。人们提出过很多种基本数据模型，其中最著名的有层次模型、网状模型、关系模型和面向对象模型。

1. 层次模型

层次模型是最早出现的数据模型，由于它采用了树状结构作为数据的组织方式，在这种结构中，每一个结点可以有多个孩子结点，但只能有一个双亲结点，这样，整体结构也是分层状的，所以称其为层次模型。层次模型数据库系统的典型代表是IBM公司的IMS数据库管理系统，该系统是1968年推出的，曾经作为大型商用数据库系统被广泛使用。

2. 网状模型

网状模型用有向图表示实体类型和实体之间的联系。网状模型的优点是记录之间的联系通过指针实现，多对多的联系容易实现，查询效率高；其缺点是编写应用程序比较复杂，程序员必须熟悉数据库的逻辑结构。由图和树的关系可知，层次模型是网状模型的一个特例。

3. 关系模型

关系模型用表格结构表达实体集，用外键表示实体之间的联系。关系模型建立在严格的数学概念基础上，概念单一、结构简单、清晰，用户易懂易用；存取路径对用户透明，从而数据独立性和安全性好，能简化数据库开发工作。其缺点主要是由于存取路径透明，查询效率往往不如非关系数据模型。

关系模型是目前应用最广泛的一种数据模型，例如，Oracle、DB2、SQL Server、Sybase 和 MySQL 等都是关系数据库系统。

4. 面向对象模型

面向对象模型是用面向对象的观点来描述现实世界实体的逻辑组织、对象之间的限制和联系等的模型。目前，已有多种面向对象数据库产品，例如，ObjectStore、Versant Developer、Suite Poet 和 Objectivity 等，但其具体的应用并不多。目前主流的方式是在现有的关系模型数据库系统中增加对对象模型的支持，如 Oracle 等。

5.2 关系数据库

关系数据库是目前应用非常广泛的数据库之一，有一套完整的理论做支持。关系模型是关系数据库的基础，由关系数据结构、关系操作集合和关系完整性规则三部分组成。本节介绍关系模型的基本概念、关系代数和关系演算以及关系规范化理论方面的内容。

5.2.1 关系的基本概念

关系数据库应用数学方法来处理数据库中的数据。最早提出将这类方法用于数据处理的是 CODASYL（数据系统语言会议）于 1962 年发表的“信息代数”一文，之后 David Child 于 1968 年在 7090 机上实现了集合论数据结构，但系统而严格地提出关系模型的是美国 IBM 公司的 E.F.Codd。

关系数据库中涉及的基本术语有：

(1) 属性 (Attribute)。在现实世界中，要描述一个事物常常取若干特征来表示，这些特征称为属性。例如，学生用学号、姓名、性别、系别、年龄、籍贯等属性来描述。

(2) 域 (Domain)。每个属性的取值范围所对应的值的集合，称为该属性的域。例如，学号的域是 6 位整型数，姓名的域是 10 位字符，性别的域为 {男, 女} 等。一般在关系数据模型中，对域还加了一个限制，所有的域都应是原子数据 (Atomic Data)。例如，整数、字符串是原子数据，而集合、记录、数组是非原子数据。

(3) 目或度 (Degree)。二维表关系 $R(D_1, D_2, \dots, D_n)$ 中的 R 表示关系的名字， n 是关系的目或度。

(4) 候选码 (Candidate Key)。若关系中的某一属性或属性组的值能唯一地标识一个元组，则称该属性或属性组为候选码。

(5) 主码 (Primary Key)。或称主键，若一个关系有多个候选码，则选定其中一个为主码。

(6) 主属性 (Prime Attribute)。包含在任何候选码中的属性称为主属性。不包含在任何候选码中的属性称为非主属性 (Nonprime Attribute)。

(7) 外码 (Foreign Key)。如果关系模式 R 中的属性或属性组非该关系的码，但它是其他关系的码，那么该属性集对关系模式 R 而言是外码。例如，客户与贷款之间的借贷联系 $c-l(c-id, loan-no)$ ，属性 $c-id$ 是客户关系中的码，所以 $c-id$ 是外码；属性 $loan-no$ 是贷款关系中的码，

所以 loan-no 也是外码。

(8) 全码 (All-key)。关系模型的所有属性组是这个关系模式的候选码, 称为全码。例如, 关系模式 $R(T, C, S)$, 属性 T 表示教师, 属性 C 表示课程, 属性 S 表示学生。假设一个教师可以讲授多门课程, 某门课程可以由多个教师讲授, 学生可以听不同教师讲授的不同课程, 那么, 要想区分关系中的每一个元组, 这个关系模式 R 的码应为全属性 T 、 C 和 S , 即 All-key。

5.2.2 关系模型

在关系模型中, 实体以及实体间的联系都是用关系来表示的。在一个给定的现实世界领域中, 所有实体及实体间的联系的关系的集合构成了一个关系数据库。关系的描述称为关系模式, 关系模式通常可以简记为 $R(A_1, A_2, \dots, A_n)$, 其中 R 为关系名, $A_1, A_2, \dots, A_n$ 为属性名。关系实际上就是关系模式在某一时刻的状态或内容。也就是说, 关系模式是型, 关系是它的值。关系模式是静态的、稳定的, 而关系是动态的、随时间不断变化的, 因为关系操作在不断地更新着数据库中的数据。但在实际应用中, 通常将关系模式和关系系统称为关系, 读者可以从上下文中加以区别。

1. 关系运算

关系代数的基本运算主要有并、差、交、笛卡儿积、投影、选择、连接和除。

(1) 并。计算两个关系在集合理论上的并集, 即给出关系 R 和 S (两者有相同元/列数), $R \cup S$ 的元组包括 R 和 S 所有元组的集合, 形式定义如下:

$$R \cup S \equiv \{t | t \in R \vee t \in S\}$$

式中 t 是元组变量 (下同)。显然, $R \cup S = S \cup R$ 。

(2) 差。计算两个关系的区别的集合, 即给出关系 R 和 S (两者有相同元/列数), $R - S$ 的元组包括 R 中有而 S 中没有的元组的集合, 形式定义如下:

$$R - S \equiv \{t | t \in R \wedge t \notin S\}$$

(3) 交。计算两个关系集合理论上的交集, 即给出关系 R 和 S (两者有相同元/列数), $R \cap S$ 的元组包括 R 和 S 相同元组的集合, 形式定义如下:

$$R \cap S \equiv \{t | t \in R \wedge t \in S\}$$

显然, $R \cap S = R - (R - S)$ 和 $R \cap S = S - (S - R)$ 成立。

(4) 笛卡儿积。计算两个关系的笛卡儿乘积, 令 R 为有 m 元的关系, S 为有 n 元的关系, 则 $R \times S$ 是 $m+n$ 元的元组的集合, 其前 m 个元素来自 R 的一个元组, 而后 n 个元素来自 S 的一个元组。形成定义如下:

$$R \times S \equiv \{t | t = \langle t_r, t_s \rangle \wedge t_r \in R \wedge t_s \in S\}$$

若 R 有 u 个元组, S 有 v 个元组, 则 $R \times S$ 有 $u \times v$ 个元组。

(5) 投影。从一个关系中抽取指明的属性 (列)。令 R 为一个包含属性 A 的关系, 则

$$\pi_A(R) \equiv \{t[A] | t \in R\}$$

(6) 选择。从关系 R 中抽取满足给定限制条件的记录, 记作

$$\sigma_F(R) \equiv \{t \mid t \in R \wedge F(t) = \text{true}\}$$

其中, F 表示选择条件, 是一个逻辑表达式 (逻辑运算符 + 算术表达式)。选择运算是从元组 (行) 的角度进行的运算。

(7) 连接 (Join)。连接分为 θ 连接、等值连接和自然连接三种。

- θ 连接。从两个关系的笛卡儿积中选取属性之间满足一定条件的元组, 记作

$$R \bowtie_{A\theta B} S \equiv \{t_r t_s \mid t_r \in R \wedge t_s \in S \wedge t_r[A] \theta t_s[B]\}$$

其中, A 和 B 分别为 R 和 S 上元数相等且可比的属性组。

- 等值连接。当 θ 为 “=” 时, 称为等值连接, 记作

$$R \bowtie_{A=B} S \equiv \{t_r t_s \mid t_r \in R \wedge t_s \in S \wedge t_r[A] = t_s[B]\}$$

- 自然连接。自然连接是一种特殊的等值连接, 它要求两个关系中进行比较的分量必须是相同的属性组, 并且在结果集中将重复属性列去掉。记作

$$R \bowtie S \equiv \{t_r t_s \mid t_r \in R \wedge t_s \in S \wedge t_r[A] = t_s[B]\}$$

(8) 除。设有关系 $R(X, Y)$ 与关系 $S(Z)$, Y 和 Z 具有相同的属性个数, 且对应属性出自相同域。关系 $R(X, Y) \div S(Z)$ 所得的商关系是关系 R 在属性 X 上投影的一个子集, 该子集和 $S(Z)$ 的笛卡儿积必须包含在 $R(X, Y)$ 中, 记为 $R \div S$, 其具体计算公式为

$$R \div S = \pi_{1,2,\dots,r-s}(R) - \pi_{1,2,\dots,r-s}((\pi_{1,2,\dots,r-s}(R) \times S) - R)$$

例如, 有关系 R 与关系 S 如表 5-1 和表 5-2 所示。

表 5-1 关系 R

$U1$	$U2$	$U3$	$U4$
a	b	c	d
a	b	e	f
c	a	c	d

表 5-2 关系 S

$U3$	$U4$
c	d
e	f

则 $R \div S$ 的求解过程为: 首先, 按除运算定义要求, 确定 X, Y, Z 属性集合。 Y 是关系 R 中的属性集合, Z 是 S 中全部属性的集合, 即 $Z = \{U3, U4\}$, 由于 $Y = Z$, 因此, $Y = \{U3, U4\}$, $X = \{U1, U2\}$ 。也就是说, $R \div S$ 结果集包含属性 $U1$ 和 $U2$ 。然后, 将关系 R 的 $U1, U2$ (共有 $\langle a, b \rangle, \langle c, a \rangle$ 两个元组) 与关系 S 做笛卡儿积操作, 结果如表 5-3 所示。

表 5-3 $R(U1, U2) \times S$

$U1$	$U2$	$U3$	$U4$
a	b	c	d
a	b	e	f
c	a	c	d
c	a	e	f

通过检查表 5-3, 可以发现元组 $\langle a, b \rangle$ 与 $S(Z)$ 的笛卡儿积被包含在 $R(X, Y)$ 中, 而元组 $\langle c, a \rangle$ 与 $S(Z)$ 的笛卡儿积有一个元组未被包含在 $R(X, Y)$ 中, 所以, 结果集中只有元组 $\langle a, b \rangle$ 。

2. 元组演算

在元组演算中,元组演算表达式简称为元组表达式,其一般形式为 $\{t|P(t)\}$,其中, t 是元组变量,表示一个元数固定的元组; $P(t)$ 是公式,在数理逻辑中也称为谓词,也就是计算机语言中的条件表达式。 $\{t|P(t)\}$ 表示满足公式 P 的所有元组 t 的集合。

在元组表达式中,公式由原子公式组成,原子公式有下列两种形式:

(1) $R(s)$,其中 R 是关系名, s 是元组变量。其含义是“ s 是关系 R 的一个元组”。

(2) $s[i]\theta u[j]$,其中 s 和 u 是元组变量, θ 是算术比较运算符, $s[i]$ 和 $u[j]$ 分别是 s 的第 i 个分量和 u 的第 j 个分量。原子公式 $s[i]\theta u[j]$ 表示“元组 s 的第 i 个分量与元组 u 的第 j 个分量之间满足 θ 运算”。例如, $t[2]<u[3]$ 表示元组 t 的第2个分量小于元组 u 的第3个分量。这个原子公式的一种简化形式是 $s[i]\theta a$ 或 $a\theta u[j]$,其中 a 为常量。例如, $t[4]=3$ 表示 t 的第4个分量等于3。

在一个公式中,如果元组变量未用存在量词“ $\exists$ ”或全称量词“ $\forall$ ”等符号定义,那么称其为自由元组变量,否则称为约束元组变量。公式的递归定义如下:

(1) 每个原子是一个公式,其中的元组变量是自由变量。

(2) 如果 P_1 和 P_2 是公式,那么, $\neg P_1$ 、 $P_1 \vee P_2$ 、 $P_1 \wedge P_2$ 和 $P_1 \rightarrow P_2$ 也是公式。

(3) 如果 P_1 是公式,那么 $(\exists s)(P_1)$ 和 $(\forall s)(P_1)$ 也都是公式。

(4) 公式中各种运算符的优先级从高到低依次为 θ 、 $\exists$ 和 $\forall$ 、 $\neg$ 、 $\wedge$ 和 $\vee$ 、 $\rightarrow$ 。在公式外还可以加括号,以改变上述优先顺序。

(5) 公式只能由上述4种形式构成,除此之外构成的都不是公式。

在元组演算的公式中,有下列4个等价的转换规则:

(1) $P_1 \wedge P_2$ 等价于 $\neg(\neg P_1 \vee \neg P_2)$ 。

(2) $P_1 \vee P_2$ 等价于 $\neg(\neg P_1 \wedge \neg P_2)$ 。

(3) $(\forall s)(P_1(s))$ 等价于 $\neg(\exists s)(\neg P_1(s))$; $(\exists s)(P_1(s))$ 等价于 $\neg(\forall s)(\neg P_1(s))$ 。

(4) $P_1 \rightarrow P_2$ 等价于 $\neg P_1 \vee P_2$ 。

关系代数表达式可以转换为元组表达式,例如, $R \cup S$ 可用 $\{t|R(t) \vee S(t)\}$ 表示, $R-S$ 可用 $\{t|R(t) \wedge \neg S(t)\}$ 表示。

5.2.3 规范化理论

设有一个关系模式 $R(SNAME, CNAME, TNAME, TADDRESS)$,其属性分别表示学生姓名、选修的课程名、任课教师姓名和任课教师地址。仔细分析一下,就会发现这个模式存在下列异常问题:

(1) 数据冗余。如果某门课程有100个学生选修,那么在 R 的关系中就要出现100个元组,这门课程的任课教师姓名和地址也随之重复出现100次。

(2) 修改异常。由于上述冗余问题,当需要修改这个教师的地址时,就要修改100个元组中的地址值,否则就会出现地址值不一致的现象。

(3) 插入异常。如果不知道听课学生名单,这个教师的任课情况和家庭地址就无法进入数据库,否则就要在学生姓名处插入空值。

(4) 删除异常。如果某门课程的任课教师要更改,那么原来任课教师的地址将随之丢失。

因此,关系模式 R 虽然只有 4 个属性,但却是性能很差的模式。产生这些异常的原因与关系模式属性值之间的联系直接相关。在关系模式 R 中,学生与课程有直接联系,教师与课程有直接联系,而教师与学生无直接联系,这就产生了关系模式 R 的异常问题。如果将 R 分解成下列两个关系模式,即 R_1 (SNAME, CNAME) 和 R_2 (CNAME, TNAME, TADDRESS),则能消除上述的存储异常现象。

1. 函数依赖与键

函数依赖是数据库的一种约束,决定了关系模式属于哪种范式。设 $R(U)$ 是属性 U 上的一个关系模式, X 和 Y 是 U 的子集, r 为 R 的任一关系,如果对于 r 中的任意两个元组 u, v ,只要有 $u[X]=v[X]$,就有 $u[Y]=v[Y]$,则称 X 函数决定 Y ,或称 Y 函数依赖于 X ,记作 $X \rightarrow Y$ 。例如,记录职工信息的属性有职工号 (EMP_NO)、职工姓名 (EMP_NMAE) 和所在部门 (DEPT),则 EMP_NO 函数决定 EMP_NMAE 和 DEPT,或者说 EMP_NMAE 和 DEPT 函数依赖于 EMP_NO,记作 $EMP_NO \rightarrow EMP_NMAE$ 和 $EMP_NO \rightarrow DEPT$ 。

在 $R(U)$ 中,如果 $X \rightarrow Y$,并且对于 X 的任何一个真子集 X' ,都有 $X' \rightarrow Y$ 不成立,则称 Y 对 X 完全函数依赖。若 $X \rightarrow Y$,但 Y 不完全函数依赖于 X ,则称 Y 对 X 部分函数依赖。在 $R(U)$ 中,如果 $X \rightarrow Y$ (Y 不是 X 的真子集),且 $Y \rightarrow X$ 不成立, $Y \rightarrow Z$,则称 Z 对 X 传递函数依赖。

关系模式的键也称为码或关键字。在关系模式中,如果有 $X \rightarrow U$ 在关系模式 $R(U)$ 上成立,并且不存在 X 的任一真子集 X' 使 $X' \rightarrow U$ 成立,那么称 X 是 R 的一个候选键。也就是说, X 值唯一决定关系中的所有元组。在关系模式中,若有多个候选键,则选一个作为主键。如果两个关系拥有公共属性(集),且公共属性在一个关系中是主键,则称公共属性是另一个关系的外键。例如,记录职工信息的属性有职工号 (EMP_NO)、职工身份证号 (EMP_CARDID)、职工姓名 (EMP_NMAE)、职工性别 (EMP_SEX) 和所在部门编号 (DEPT_NO),则在此关系中,EMP_NO 或 EMP_CARDID 是候选键,也可以是本关系的主键。一个关系的候选键有多个,但主键只能有一个。通常在候选键中选一个作为主键。

求关系模式的候选键是进行范式界定的基础,也是系统分析师应该掌握的基本技能。使用候选键的定义来求解一个简单关系模式的候选键尚能应对,但面对复杂一些的关系模式,这种方法就不管用了。在此,引入一种求候选键的快捷方法,即图示法。使用图示法求候选键,主要有两个步骤:

(1) 将关系模式的函数依赖关系用有向图的方式表示,其中顶点表示属性,弧表示属性之间的依赖关系。

(2) 找出入度为 0 的属性集,并以该属性集为起点,尝试遍历有向图,若能正常遍历图中所有结点,则该属性集即为关系模式的候选键;若入度为 0 的属性集不能遍历图中所有结点,则需要尝试性地将一些中间顶点(既有入度,也有出度的顶点)并到入度为 0 的属性集中,直至该集合能遍历所有顶点,则该集合为候选键。

例如,给定关系 $R(A_1, A_2, A_3, A_4)$ 上的函数依赖集 $F=\{A_1 \rightarrow A_2, A_3 \rightarrow A_2, A_2 \rightarrow A_3,$

$A_2 \rightarrow A_4\}$ ，现在要求 R 的候选键。需要针对函数依赖集画出有向图，如图 5-2 所示。

从图 5-2 中找出入度为 0 的顶点，即 A_1 。通过尝试，可以发现从 A_1 出发可以遍历所有顶点，因此， R 的候选键为 A_1 。

2. 范式

为了设计一个好的数据库，人们定义了一些好的关系模式标准，称它们为规范的关系模式或范式 (Normal Form, NF)。目前共定义了多个范式，分别为 1NF、2NF、3NF、BCNF、4NF 和 5NF。但在实际应用中，一般只要达到 3NF。

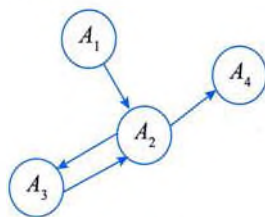


图 5-2 有向图

(1) 第一范式 (1NF)。在关系模式 R 中，当且仅当所有属性只包含原子值，即每个分量都是不可再分的数据项，则称 R 满足 1NF。例如，如表 5-4 所示的教师职称情况关系就不满足 1NF。原因在于，该关系模式中的“高级职称人数”不是一个原子属性，若将其拆分为“教授”和“副教授”两个属性，则满足 1NF。

表 5-4 教师职称情况关系表

系 名 称	高级职称人数	
	教 授	副 教 授
计算机系	6	10
电子系	3	5

(2) 第二范式 (2NF)。当且仅当关系模式 R 满足 1NF，且每个非键属性 (即不属于任何候选键的属性，也称为非主属性) 完全依赖于候选键时，则称 R 满足 2NF。例如，有选课关系模式 $SC(Sno, Cno, Grade, Credit)$ ，其中， $(Sno, Cno) \rightarrow Grade$ ， $Cno \rightarrow Credit$ 。因此， SC 的候选键为 (Sno, Cno) 。这样， $Cno \rightarrow Credit$ 就构成了 $Credit$ 对候选键 (Sno, Cno) 的部分函数依赖。因此， SC 不满足 2NF。若要将 SC 转化为 2NF，可以将它拆分为 $SC1(Sno, Cno, Grade)$ 和 $SC2(Cno, Credit)$ 。

(3) 第三范式 (3NF)。当且仅当关系模式 R 满足 2NF，且 R 中消除了非键属性传递依赖于候选键时，则称 R 满足 3NF。例如，学生关系 $S(Sno, Sname, Dno, Dname, Location)$ 的各属性分别代表学号、姓名、所在系、系名称和系地址，其数据如表 5-5 所示。

表 5-5 关系 S 的数据

Sno	Sname	Dno	Dname	Location
S01	张三	D01	计算机系	1 号楼
S02	李四	D01	计算机系	1 号楼
S03	王五	D01	计算机系	1 号楼
S04	赵六	D02	信息系	2 号楼
...	...	...	...	...

从各属性之间的联系可以判断出 S 的函数依赖有 $Sno \rightarrow (Sname, Dno, Dname, Location)$, $Dno \rightarrow (Dname, Location)$ 。显然, Sno 为候选键。在函数依赖中有 $Sno \rightarrow Dno \rightarrow Dname$ 与 $Sno \rightarrow Dno \rightarrow Location$, 这便是传递函数依赖。由于 $Dname$ 与 $Location$ 为非键属性, 同时传递依赖于候选键, 因此, 关系模式 S 不满足 3NF。若要使 S 满足 3NF, 需要将其拆分为 $S1(Sno, Sname, Dno)$ 和 $S2(Dno, Dname, Location)$ 。

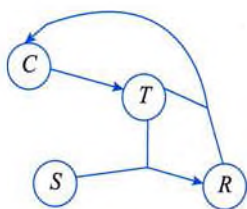


图 5-3 关系模式 P 的函数依赖图

(4) BCNF。如果关系模式 R 满足 1NF, 且 R 中没有属性传递依赖于候选键时, 则称 R 满足 BCNF。例如, 有关系模式 $P(C, S, T, R)$, 其函数依赖集 $F=\{C \rightarrow T, ST \rightarrow R, TR \rightarrow C\}$, 现在需要判断 P 是否满足 BCNF。先画出相应的函数依赖图, 如图 5-3 所示。

对图 5-3 进行分析, 可以得知, P 的候选键有 (S, T) 和 (S, C) , 键属性 (属于某个候选键的属性, 也称为主属性) 有 S, T 和 C , 非键属性只有 R 。但此时由于属性之间的联系错综复杂, 要界定关系模式是否存在传递函数依赖并不容易。为了准确地界定关系模式是否为 BCNF, 需要引入一些另外的判别方法。例如, 一个 BCNF 的关系模式必须同时满足以下条件: 所有非键属性对每个候选键都是完全函数依赖的, 所有的键属性对每个不包含它的候选键也是完全函数依赖的; 没有任何属性完全函数依赖于非键属性, 即每个函数依赖的左部都必须包含候选键。在关系模式 P 中, 由于有 $C \rightarrow T$, 而 C 不包含候选键, 因此, P 不满足 BCNF。

(5) 第四范式 (4NF)。第四范式是 BCNF 的推广, 是针对有多值依赖的关系模型所定义的规范化形式。关系模式 $R(U, F)$ 满足 1NF, X, Y 是 U 的非空子集, $Z=U-X-Y$ 也非空, 若任取一组值对 (x, z) , 都可决定一组 y 值, 且此决定与 z 值无关, 就称 Y 多值依赖于 X , 记做 $X \twoheadrightarrow Y$ 。关系模式 R 满足 1NF, 若对任一多值依赖 $X \twoheadrightarrow Y$, X 必包含 R 的候选键, 称 R 满足 4NF。例如, 如表 5-6 所示的关系 $QY(ypm, bm, sccj)$ 。

表 5-6 关系 QY

用品名 (ypm)	部门 (bm)	生产厂家 (sccj)
办公桌	生产经营部	美时办公用品公司
办公桌	总经理办公室	华鹤家具有限公司
办公椅	生产经营部	美时办公用品公司
办公椅	经理办公室	华鹤家具有限公司
办公椅	计划部	华鹤家具有限公司

表 5-6 是实际工作中常见的登记表, 抛开是否规范不说, 这样的登记表一目了然。但从规范化的角度来看, 对 ypm 的一个值, 不论 $sccj$ 取什么值, 总有一组确定的 bm 与之对应, 所以有 $ypm \rightarrow bm$ 。同样, 有 $ypm \rightarrow sccj$ 。 QY 是全码关系 (即所有的属性合在一起形成候选键), 这说明 QY 不满足 4NF。可用分解法消除不满足 4NF 的多值依赖, 解决办法是将 QY 分解为 $QY1(ypm, bm)$ 和 $QY2(ypm, sccj)$ 。

3. 关系模式分解

如果某关系模式存在修改异常等问题,则可通过分解该关系模式来解决问题。将一个关系模式分解成几个子关系模式,需要考虑的是该分解是否保持函数依赖,是否是无损联接。

无损联接分解的形式定义如下:设 R 是一个关系模式, F 是 R 上的一个函数依赖集。 R 分解成数据库模式 $\delta = \{R_1, \dots, R_k\}$ 。如果对 R 中每个满足 F 的关系 r 都有下式成立:

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_k}(r)$$

则称分解 δ 相对于 F 是无损联接分解,否则称为损失联接分解。

要根据上述定义来判断一个分解是否是无损联接,这是一件很困难的事情,下面是一个很有用的无损联接分解判定定理:设 $\rho = \{R_1, R_2\}$ 是 R 的一个分解, F 是 R 上的函数依赖集,那么分解 ρ 相对于 F 是无损联接分解的充要条件是 $(R_1 \cap R_2) \rightarrow (R_1 - R_2)$ 或 $(R_1 \cap R_2) \rightarrow (R_2 - R_1)$ 。要注意的是,这两个条件只要有任意一个条件成立就可以了。

设数据库模式 $\delta = \{R_1, \dots, R_k\}$ 是关系模式 R 的一个分解, F 是 R 上的函数依赖集, δ 中每个模式 R_i 上的函数依赖集是 F_i 。如果 $\{F_1, F_2, \dots, F_k\}$ 与 F 是等价的(即相互逻辑蕴涵),则称分解 δ 保持函数依赖。如果分解不能保持函数依赖,则 δ 的实例上的值就可能有违反函数依赖的现象。

5.3 数据库控制功能

要想使数据库中的数据达到应用的要求,必须对其进行各种控制,这就是 DBMS 的控制功能,包括并发控制、性能优化、数据库的完整性和安全性,以及数据备份与恢复等。这些技术虽然给人们的感觉是边缘性技术,但对 DBMS 的应用而言却是至关重要的。

5.3.1 并发控制

在多用户共享系统中,许多事务可能同时对同一数据进行操作,称为并发操作。此时,DBMS 的并发控制子系统负责协调并发事务的执行,保证数据库的完整性不受破坏,同时,避免用户得到不正确的数据。

1. 事务的基本概念

DBMS 运行的基本工作单位是事务,事务是用户定义的一个数据库操作序列,这些操作序列要么全做,要么全不做,是一个不可分割的工作单位。事务具有以下特性:

(1) 原子性 (Atomicity)。事务是数据库的逻辑工作单位,事务的原子性保证事务包含的一组更新操作是原子不可分的,也就是说,这些操作是一个整体,不能部分地完成。

(2) 一致性 (Consistency)。一致性是指使数据库从一个一致性状态变到另一个一致性状态。例如,在转账的操作中,各账户金额必须平衡。一致性与原子性是密切相关的,一致性在逻辑上不是独立的,它由事务的隔离性来表示。

(3) 隔离性 (Isolation)。隔离性是指一个事务的执行不能被其他事务干扰,即一个事务内

部的操作及使用的数据对并发的其他事务是隔离的，并发执行的各个事务之间不能互相干扰。它要求即使有多个事务并发执行，但看上去每个事务是按串行调度执行一样。这一性质也称为可串行性，也就是说，系统允许的任何交错操作调度等价于一个串行调度。

(4) 持久性 (Durability)。持久性也称为永久性，是指事务一旦提交，改变就是永久性的，无论发生何种故障，都不应该对其有任何影响。

事务的原子性、一致性、隔离性和持久性通常统称为 ACID 特性。

2. 数据不一致问题

数据库的并发操作会带来一些数据不一致问题，例如，丢失修改、读“脏数据”和不可重复读等。

(1) 丢失修改。事务 A 与事务 B 从数据库中读入同一数据并修改，事务 B 的提交结果破坏了事务 A 提交的结果，导致事务 A 的修改被丢失。例如，有 T1、T2 两个事务，其执行顺序如表 5-7 所示，则“③ A=A-5，写回”操作会被“④ A=A-8，写回”操作覆盖掉，“③ A=A-5，写回”将不起任何作用。

表 5-7 丢失修改的实例

T1	T2
①读 A=10	
②	读 A=10
③ A=A-5，写回	
④	A=A-8，写回

(2) 读“脏数据”。事务 A 修改某一数据，并将其写回磁盘，事务 B 读取同一数据后，事务 A 由于某种原因被撤销，这时事务 A 已修改过的数据恢复原值，事务 B 读到的数据就与数据库中的数据不一致，是不正确的数据，称为“脏数据”。例如，有 T1、T2 两个事务，其执行顺序如表 5-8 所示，则 T2 中“读 A=70”就是读的“脏数据”。

表 5-8 读“脏数据”的实例

T1	T2
①读 A=20 A=A+50 写回 70	
②	
③ ROLLBACK A 恢复为 20	读 A=70

(3) 不可重复读。不可重复读是指事务 A 读取数据后，事务 B 执行了更新操作，事务 A 使用的仍是更新前的值，造成了数据不一致性。例如，有 T1、T2 两个事务，其执行顺序如表 5-9 所示。

表 5-9 不可重复读的实例

T1	T2
①读 A=20 读 B=30 求和 =50 ②	
③读 A=70 读 B=30 求和 =100 (验算不对)	读 A=20 A=A+50 写 A=70

在表 5-9 中, T1 事务为了确保其重要计算无误, 所以采用了验算的方式, 两次独立取出数据并运算, 最后进行验算 (即比较两次运算结果是否相同)。在此处, 虽然两次计算都没错, 但由于在两次操作之间的时间间隔中, T2 对数据进行了修改, 导致验算结果不正确, 这就是不可重复读问题。

3. 封锁协议

处理并发控制的主要方法是采用封锁技术, 主要有两种封锁, 分别是 X 封锁和 S 封锁。

(1) 排他型封锁 (X 封锁)。如果事务 T 对数据对象 A (可以是数据项、元组和数据集, 以至整个数据库) 实现了 X 封锁, 那么只允许事务 T 读取和修改数据 A, 其他事务要等事务 T 解除 X 封锁以后, 才能对数据 A 实现任何类型的封锁。可见, X 封锁只允许一个事务独锁某个数据, 具有排他性。

(2) 共享型封锁 (S 封锁)。X 封锁只允许一个事务独锁和使用数据, 要求太严, 需要适当从宽。例如, 可以允许并发读, 但不允许修改, 这就产生了 S 封锁的概念。S 封锁的含义是, 如果事务 T 对数据 A 实现了 S 封锁, 那么允许事务 T 读取数据 A, 但不能修改数据 A, 在所有 S 封锁解除之前, 决不允许任何事务对数据 A 实现 X 封锁。

在多个事务并发执行的系统中, 主要采取封锁协议来进行处理。常见的封锁协议如下:

(1) 一级封锁协议。事务 T 在修改数据 R 之前必须先对其加 X 锁, 直到事务结束才释放。一级封锁协议可防止丢失修改, 并保证事务 T 是可恢复的, 但不能保证可重复读和不读“脏数据”。

(2) 二级封锁协议。一级封锁协议加上事务 T 在读取数据 R 之前先对其加 S 锁, 读完后即可释放 S 锁。二级封锁协议可防止丢失修改, 还可防止读“脏数据”, 但不能保证可重复读。

(3) 三级封锁协议。一级封锁协议加上事务 T 在读取数据 R 之前先对其加 S 锁, 直到事务结束才释放。三级封锁协议可防止丢失修改、读“脏数据”, 且能保证可重复读。

(4) 两段锁协议。所有事务必须分两个阶段对数据项加锁和解锁。其中, 扩展阶段是在对任何数据进行读、写操作之前, 首先要申请并获得对该数据的封锁; 收缩阶段是在释放一个封锁之后, 事务不能再申请和获得任何其他封锁。若并发执行的所有事务均遵守两段锁协议, 则

对这些事务的任何并发调度策略都是可串行化的。遵守两段锁协议的事务可能发生死锁。

显然，使用封锁技术来解决并发控制问题存在一个封锁粒度问题。所谓封锁粒度，是指被封锁数据对象的大小，在关系数据库中封锁粒度有属性值、属性值集、元组、关系、某索引项（或整个索引）、整个关系数据库、物理页（块）等几种。封锁粒度小则并发性高，但开销大；封锁粒度大则并发性低，但开销小。综合平衡照顾不同需求，以合理选取适当的封锁粒度是很重要的。

4. 死锁问题

采用封锁的方法虽然可以有效防止数据的不一致性，但封锁本身也会产生一些麻烦，最主要的就是死锁问题。死锁是指多个用户申请不同封锁，由于申请者均拥有一部分封锁权，而又需等待其他用户拥有的部分封锁而引起的永无休止的等待。

5.3.2 数据库的完整性

数据库的完整性是指数据库中数据的正确性和相容性。数据库的完整性由各种各样的完整性约束来保证，完整性约束可以通过 DBMS 或应用程序来实现，基于 DBMS 的完整性约束作为关系模式的一部分存入数据库中。

1. 完整性约束条件

完整性约束条件是指对数据库中数据本身的某些语法或语义限制、数据之间的逻辑约束，以及数据变化时应遵守的规则等。所有这些约束条件一般均以谓词逻辑形式表示，即以具有真假值的原子公式和命题连接词（并且、或者、否则）所组成的逻辑公式表示。完整性约束条件的作用对象可以是关系、元组或属性三种。数据的完整性约束条件一般在关系模式中给出，并在运行时做检查，当不满足条件时立即向用户通报，以便采取措施。

数据库中数据的语法、语义限制与数据之间的逻辑约束称为静态约束，它反映了数据及其之间的固有逻辑特性，是最重要的一类完整性约束。静态约束包括静态属性级约束（对数据类型的约束、对数据格式的约束、对取值范围或取值集合的约束、对空值的约束以及其他约束）、静态元组约束和静态关系约束（实体完整性约束、参照完整性约束、函数依赖约束、统计约束）。

数据库中的数据变化应遵守的规则称为数据动态约束，它反映了数据库状态变迁的约束。动态约束包括动态属性级约束（修改属性定义时的约束、修改属性值时的约束）、动态元组约束和动态关系约束。

完整性控制机制应该具有定义功能和检查功能，定义功能提供定义完整性约束条件的机制，检查功能检查用户发出的操作请求是否违背了完整性约束条件。如果发现用户的操作请求违背了约束条件，则采取一定的动作来保证数据的完整性。

2. 实体完整性

实体完整性要求主键中的任一属性不能为空，所谓空值是“不知道”或“无意义”的值。之所以要保证实体完整性，主要是因为，在关系中，每个元组的区分是依据主键值的不同，若主

键值取空值, 则不能表明该元组的存在。例如, 对于学生关系 $S(Sno, Sname, Ssex)$, 其主键为 Sno , 在插入某个元组时, 就必须要求 Sno 不能为空。更加严格的 DBMS 还要求 Sno 不能与已经存在的某个元组的 Sno 相同。

3. 参照完整性

若基本关系 R 中含有与另一基本关系 S 的主键 PK 相对应的属性组 FK (FK 称为 R 的外键), 则参照完整性要求, 对 R 中的每个元组在 FK 上的值必须是 S 中某个元组的 PK 值, 或者为空值。参照完整性的合理性在于, R 中的外键只能对 S 中的主键引用, 不能是 S 中主键没有的值。例如, 对于学生关系 $S(Sno, Sname, Ssex)$ 和选课关系 $C(Sno, Cno, Grade)$ 两个关系, C 中的 Sno 是外键, 它是 S 的主键, 若 C 中出现了某个 S 中没有的 Sno , 即某个学生还没有注册, 却已有了选课记录, 这显然是不合理的。

4. 用户定义的完整性

实体完整性和参照完整性适用于任何关系型 DBMS。除此之外, 不同的数据库系统根据其应用环境的不同, 往往还需要一些特殊的约束条件。用户定义的完整性就是针对某一具体数据库的约束条件, 反映某一具体应用所涉及的数据必须满足的语义要求。

如果在一条语句执行完后立即检查, 称为立即执行约束; 如果在整个事务执行结束后再进行检查, 则称延迟执行约束。完整性规则的五元组表示为 (D, O, A, C, P) , 其中, D 表示约束作用的数据对象, O 表示触发完整性检查的数据库操作, A 表示数据对象必须满足的断言或语义约束, C 表示选择 A 作用的数据对象值的谓词, P 表示违反完整性规则时触发的过程。

5. 触发器

触发器是在关系型 DBMS 中应用得比较多的一种完整性保护措施, 其功能比完整性约束要强得多。一般而言, 在完整性约束功能中, 当系统检查出数据中有违反完整性约束条件时, 仅给出必要提示以通知用户。而触发器的功能则不仅起到提示作用, 还会引起系统自动进行某些操作, 以消除违反完整性约束条件所引起的负面影响。

所谓触发器, 其抽象的含义是一个事件的发生必然触发 (或导致) 另外一些事件的发生, 其中, 前面的事件称为触发事件, 后面的事件称为结果事件。触发事件一般即为完整性约束条件的否定, 而结果事件即为一组操作, 用以消除触发事件所引起的不良影响。目前, 数据库中的事件一般表示为数据的插入、修改、删除等操作。触发器除了有完整性保护功能外, 还有安全性保护功能。

5.3.3 数据库的安全性

就整个信息系统的安全而言, 数据的安全是最重要的。数据库系统的安全性在技术上依赖于两种方式: 一种是 DBMS 本身提供的用户身份识别、视图、使用权限控制和审计等管理措施, 大型 DBMS 均有此功能; 另一种就是靠应用程序来实现对数据库访问进行控制和管理, 也就是说, 数据的安全控制由应用程序里面的代码来实现。目前, 一些大型 DBMS 都提供了一些技术手段来保证数据的安全, 如表 5-10 所示。

表 5-10 数据库安全性措施

措 施	说 明
用户标识和鉴别	最外层的安全保护措施，可以使用用户账户、口令和随机数检验等方式
存取控制（数据授权）	对用户进行授权，包括操作类型（例如，查找、更新或删除等）和数据对象的权限
密码存储和传输	对远程终端信息用密码传输
视图的保护	通过视图的方式进行授权
审计	使用一个专用文件或数据库，自动将用户对数据库的所有操作记录下来

下面主要介绍用户标识和鉴别、数据授权、视图以及审计与跟踪等措施。

1. 用户标识和鉴别

用户的身份认证是用户使用 DBMS 系统的第一个环节，是系统提供的最外层保护。进行用户标识和鉴别的常用方式有口令认证和强身份认证。

强身份认证指的是在网络环境下，客户端到 DBMS 服务器可能经过多个环节，在身份认证期间，用户的信息和口令可能会经过很多不安全的节点（例如，路由器和服务器）而被信息的窃听者窃取。强身份认证过程使认证可以结合信息安全领域一些更深入的技术保障措施，来强化用户身份的鉴别，例如，可以与数字证书、智能卡 and 用户指纹识别等多种身份识别技术相结合。

2. 数据授权

当用户通过身份认证以后，并不是所有的用户都能操作所有的数据，要分不同的用户角色来区别对待，例如，普通用户只能查看自己的个人信息，而 DBA 则可以查看所有用户的信息。要达到这一效果，需要对不同用户角色进行不同级别的数据授权。

一般可以将权限角色分为三类：数据库登录权限类、资源管理权限类和 DBA 权限类。不同的 DBMS 可能对用户角色的定义不尽相同，权限划分的细致程度也远超过上面三种基本的类型。同一类功能操作权限的用户，对数据库中的数据对象管理和使用的范围也可能是不同的，因此，DBMS 除了要提供基于功能角色的操作权限控制外，还提供对数据对象的访问控制，访问控制可以根据对控制用户访问数据对象的范围（或称粒度）从大到小分为 4 个层次，分别是数据库级、关系级、元组级和属性级。

3. 视图

视图可以被看成是虚拟关系或存储查询，可通过视图访问的数据不作为独特的对象存储在数据库内，数据库内存储的是 SELECT 语句。SELECT 语句的结果集构成视图所返回的虚拟关系。用户可以用引用关系所使用的方法，在 SQL 语句中通过引用视图名称来使用虚拟关系。

4. 审计与跟踪

如果身份认证是一种事前的防范措施，审计则是一种事后监督的手段。跟踪也是 DBMS 提供的监视用户动作的功能，然而，审计和跟踪是两个不同的概念，主要是两者的目的不同。跟

踪主要是满足系统调试的需要,捕捉到的用户行为记录往往只用于分析,而并不长久地保存,而审计作为一种安全检查的措施,会将系统的运行状况和用户访问数据库的行为以日志形式记录并保存下来,这种日志往往作为稽查用户行为的一种证据。

5.3.4 备份与恢复技术

前面详细介绍了保证数据库安全性的技术和措施,但不管用多么高明的手段,总是难以避免安全事故的发生。安全事故的发生有可能是由于人为因素,也可能是由于硬件设备的故障,甚至是自然灾害。因此,需要备份与恢复技术来进一步保障数据的安全,即当数据被破坏后,在一定时间内将数据库调整到破坏前的状态。

数据库备份有多种分类方式。按备份的实现方式,可分为物理备份与逻辑备份,而物理备份又可以分为冷备份与热备份。按备份的数据量情况,可分为完全备份、增量备份与差异备份。其中,完全备份是指将整个数据库中的数据进行备份;增量备份是指备份上一次备份(包括完全备份、增量备份和差异备份)后发生变化的数据;差异备份是指备份上一次完全备份后发生变化的所有数据。由于备份方式存在多样性,因此,制定一个合适的可操作的备份和恢复策略至关重要,其基本原则是保证数据丢失得尽量少或完全不丢失,且备份和恢复时间尽量短,保证系统最大的可用性。

1. 物理备份

物理备份是在操作系统层面对数据库的数据文件进行备份,可分为冷备份和热备份两种。冷备份也称为静态备份,是将数据库正常关闭,在停止状态下,将数据库的文件全部备份(复制)下来。当数据库发生故障时,将数据文件复制回来进行恢复。冷备份是数据库备份中最快和最安全的方法。热备份也称为动态备份,是利用备份软件,在数据库正常运行的状态下,将数据库中的数据文件备份下来。冷备份与热备份的优缺点如表 5-11 所示。

表 5-11 冷备份与热备份优缺点比较

备份方式	优 点	缺 点
冷备份	非常快速的备份方法(只需复制文件);容易归档(简单复制即可);容易恢复到某个时间点上(只需将文件再复制回去);能与归档方法相结合,做数据库“最佳状态”的恢复;低度维护,高度安全	单独使用时,只能提供到某一时间点上的恢复;在实施备份的全过程中,数据库必须要做备份而不能做其他工作;若磁盘空间有限,只能复制到磁带等其他外部存储设备上,速度会很慢;不能按表或按用户恢复
热备份	可在表空间或数据库文件级备份,备份的时间短;备份时数据库仍可使用;可达到秒级恢复(恢复到某一时间点上);可对几乎所有数据库实体做恢复;恢复是快速的	不能出错,否则后果严重;若热备份不成功,所得结果不可用于时间点的恢复;困难于维护,所以要特别小心,不允许“以失败告终”

为了提高物理备份的效率,通常将完全备份、增量备份和差异备份三种备份方式相组合。一般来说,一个备份周期通常由一个完全备份和多个增量、差异备份组成。由于增量备份或差异备份导出的数据少,所需要的时间也较少。

2. 逻辑备份

逻辑备份是指利用 DBMS 自带的工具软件备份和恢复数据库的内容，例如，Oracle 的导出工具为 exp，导入工具为 imp，可以按照表、表空间、用户和全库 4 个层次备份和恢复数据；Sybase 的全库备份命令是 dump database，全库恢复命令是 load database，还可利用 BCP 命令来备份和恢复指定表。

在数据库容量不大的情况下，逻辑备份是一个非常有效的手段，既简单又方便。但随着数据量越来越大，甚至高达 TB 级，利用逻辑备份来恢复数据库已力不从心，速度很慢。针对大型数据库的备份和恢复，一般结合磁带库或光盘库，采用物理备份方式。

3. 日志文件

事务日志是针对数据库改变所做的记录，它可以记录针对数据库的任何操作，并将记录结果保存在独立的文件中。这种文件就称为日志文件。对于任何一个事务，事务日志都有非常全面的记录，根据这些记录可以将数据文件恢复成事务前的状态。从事务动作开始，事务日志就处于记录状态，事务执行过程中对数据库的任何操作都记录在内，直到用户提交或回滚后才结束记录。

日志文件是用来记录对数据库每一次更新活动的文件，在热备份方式中，必须建立日志文件，后援副本和日志文件综合起来才能有效地恢复数据库；在冷备份方式中，也可以建立日志文件，当数据库毁坏后，可重新装入后援副本，将数据库恢复到备份结束时刻的正确状态，然后利用日志文件，将已完成的事务进行重做处理，对故障发生时尚未完成的事务进行撤销处理。这样，不必重新运行那些已完成的事务程序就可将数据库恢复到故障前某一时刻的正确状态。

事务在运行过程中，系统将事务开始、事务结束（包括 COMMIT 和 ROLLBACK），以及对数据库的插入、删除和修改等每个操作作为一个登记记录存放到日志文件中。每个记录包括的主要内容有执行操作的事务标识、操作类型、更新前数据的旧值（对插入操作而言此项为空值）、更新后的新值（对删除操作而言此项为空值）。登记的次序严格按并事务操作执行的时间次序，同时遵循“先写日志文件”的规则。写一个修改到数据库中和写一个表示这个修改的日志记录到日志文件中是两个不同的操作，有可能在这两个操作之间发生故障，即这两个写操作只完成了一个，如果先写了数据库修改，而在日志记录中没有登记这个修改，则以后就无法恢复这个修改了。因此，为了安全，应该先写日志文件，即首先将修改记录写到日志文件上，然后再写数据库的修改。

4. 数据恢复

将数据库从错误状态恢复到某一个已知的正确状态的功能，称为数据库的恢复。数据恢复的基本原理就是冗余，建立冗余的方法有数据备份和登录日志文件等。可根据故障的不同类型，采用不同的恢复策略。

（1）事务故障的恢复。事务故障的恢复是由系统自动完成的，对用户是透明的（不需要 DBA 的参与）。其步骤如下：反向扫描日志文件，查找该事务的更新操作；对该事务的更新操作执行逆操作；继续反向扫描日志文件，查找该事务的其他更新操作，并做同样的处理；如此处理下去，直至读到此事务的开始标记，事务故障恢复完成。

(2) 系统故障的恢复。系统故障的恢复在系统重新启动时自动完成,不需要用户干预。其步骤如下:正向扫描日志文件,找出在故障发生前已经提交的事务,将其事务标识记入重做(Redo)队列。同时找出故障发生时尚未完成的事务,将其事务标识记入撤销(Undo)队列。对撤销队列中的各个事务进行撤销处理:反向扫描日志文件,对每个Undo事务的更新操作执行逆操作。对重做队列中的各个事务进行重做处理:正向扫描日志文件,对每个Redo事务重新执行日志文件登记的操作。

(3) 介质故障与病毒破坏的恢复。介质故障与病毒破坏的恢复步骤如下:装入最新的数据库后援副本,使数据库恢复到最近一次备份时的一致性状态;从故障点开始反向扫描日志文件,找出已提交事务标识并记入Redo队列;从起始点开始正向扫描日志文件,根据Redo队列中的记录,重做已完成的任务,将数据库恢复至故障前某一时刻的一致状态。

(4) 有检查点的恢复技术。检查点记录的内容可包括建立检查点时刻所有正在执行的事务清单,以及这些事务最近一个日志记录的地址。采用检查点的恢复步骤如下:从重新开始文件中找到最后一个检查点记录在日志文件中的地址,由该地址在日志文件中找到最后一个检查点记录;由该检查点记录得到检查点建立时所有正在执行的事务清单队列(A);建立重做队列(R)和撤销队列(U),将A队列放入U队列中,R队列为空;从检查点开始正向扫描日志文件,若有新开始的事务T1,则将T1放入U队列,若有提交的事务T2,则将T2从U队列移到R队列,直至日志文件结束;对U队列的每个事务执行Undo操作,对R队列的每个事务执行Redo操作。

5.3.5 数据库性能优化

通常,对一个集中式数据库的性能进行优化,可以从硬件升级、数据库设计、索引优化策略和查询优化等方面入手。

1. 硬件升级

要提升数据库的运行速度,最直接的方式就是硬件升级,涉及的硬件包括处理器、内存、磁盘子系统和网络。

处理器的升级主要可以考虑用更高频率的处理器代替现有频率较低的处理器,也可以将单处理器的计算机系统升级为多处理器系统。

内存的升级主要是容量的扩充,当容量扩充以后,数据库服务器可以将更多的数据保存在缓冲区,以减少磁盘I/O操作,从而提升数据库的整体性能。

磁盘子系统的性能提升主要体现在两个方面:第一,采用高速磁盘系统替代速度较低的磁盘系统,以减少读盘等待时间,提高响应速度;第二,合理分布磁盘I/O到多个设备上,以减少资源竞争,提高并行操作能力。

网络方面的升级主要是对带宽的升级。

2. 数据库设计

在数据库设计阶段就可以着手考虑性能优化问题。对数据库进行设计优化,主要可以从逻辑设计和物理设计两个方面入手。

根据前面的介绍,数据库的规范化程度越高,数据库中的冗余信息就越少。然而,规范化

同时又引入了新的问题，规范化使得关系模式不断被拆解，这样关系模式之间的结构变得越来越复杂，在使用数据时频繁执行连接操作，而连接操作是最耗时的，是数据库性能的制约因素。因此，从某种意义上来说，非规范化（反规范化）可以改善系统的性能。在进行数据库设计时，可以考虑合理增加冗余属性，以提升系统性能，常用的措施如下：

（1）将常用的计算属性（例如，总计和最大值等）存储到数据库实体中。

（2）重新定义实体，以减少外部属性数据或行数据的开支。

（3）将关系进行水平或垂直分割，以提升并行访问度。

数据库逻辑结构的设计固然重要，但物理设计也不可忽视，将数据放在不同的物理位置，有时能对性能提升起到非常关键的作用。例如，可以遵循以下准则：

（1）与每个属性相关的数据类型应该反映数据所需的最小存储空间，特别是对于被索引的属性更是如此。例如，能使用 `smallint` 类型的就不要用 `integer` 类型，这样，索引字段可以被更快地读取，而且可以在一个数据页上放置更多的数据行，就相应地减少了 I/O 操作。

（2）将一个频繁使用的关系模式分割开，并放在两个单独的智能型磁盘控制器的数据库设备上，这样也可以提高性能。因为有多个磁头在查找，数据分离也能提高性能。

（3）将数据库中文本或图像属性的数据存放在一个单独的物理设备上，也可以提高性能。如果使用专用的智能型控制器，就能进一步提高性能。

3. 索引优化策略

索引是提高数据库查询速度的利器，而数据库查询往往是数据库系统中最频繁的操作，因此，索引的建立与选择对数据库性能优化具有重大意义。索引的建立与选择可遵循以下准则：

（1）建立索引时，应选用经常作为查询，而不常更新的属性。避免对一个经常被更新的属性建立索引，因为这样会严重影响性能。

（2）一个关系上的索引过多会影响 `UPDATE`、`INSERT` 和 `DELETE` 的性能，因为关系一旦进行更新，所有的索引都必须跟着做相应的调整。

（3）尽量分析出每个重要查询的使用频率，这样可以找出使用最多的索引，然后可以先对这些索引进行适当的优化。

（4）对于数据量非常小的关系不必建立索引，因为对于数据量小的关系而言，关系扫描往往更快，而且消耗的系统资源更少。

4. 查询优化

查询优化也称为应用程序优化，它是数据库性能优化的最后一个环节，同时也是最重要的一个环节。查询语句的构造不当，会使之前的优化功亏一篑。SQL 语句优化的策略很多，例如，建立物化视图或尽可能减少多表查询；以不相干子查询替代相干子查询；只检索需要的属性；用带 `IN` 的条件子句等价替换 `OR` 子句；经常提交（`COMMIT`），以尽早释放锁等。

5.4 数据库设计与建模

数据库设计是指对一个给定的应用环境，提供一个确定最优的数据模型与处理模式的逻辑

设计, 以及一个确定数据库存储结构与存取方法的物理设计, 建立起能反映现实世界信息和信息联系及满足用户数据要求和加工要求, 以能够被某个 DBMS 所接受, 同时能实现系统目标, 并有效存取数据的数据库。

5.4.1 数据库设计阶段

基于数据库系统生命周期的数据库设计可分为如下 5 个阶段: 规划、需求分析、概念设计、逻辑设计和物理设计。

1. 规划

规划阶段的主要任务是进行建立数据库的必要性及可行性分析, 确定数据库系统在企业 and 信息系统中的地位, 以及各个数据库之间的联系。有关系统规划方面的知识将在第 10 章中详细介绍。

2. 需求分析

需求分析的目标是通过调查研究, 了解用户的数据和处理要求, 并按一定格式整理成需求说明书。需求说明书包括数据库所涉及的数据、数据的特征、使用频率和数据量的估计, 例如, 数据名、属性及其类型、主键属性、保密要求、完整性约束条件、更改要求、使用频率和数据量估计等。这些关于数据的数据称为元数据。在设计大型数据库时, 这些数据通常由数据字典来管理。用数据字典管理元数据, 有利于避免数据的重复或重名, 以保持数据的一致性。同时, 有利于提高数据库设计的质量, 减轻设计者的负担。有关需求分析的知识将在第 11 章中详细介绍。

3. 概念设计

概念设计也称为概念结构设计, 其任务是在需求分析阶段产生的需求说明书的基础上, 按照特定的方法将它们抽象为一个不依赖于任何 DBMS 的数据模型, 即概念模型。概念模型使设计人员的注意力能够从复杂的实现细节中解脱出来, 而只集中在最重要的数据的组织结构和处理模式上。为保证所设计的概念模型能正确、完全地反映用户的数据及其相互关系, 便于进行所要求的各种处理, 在概念设计阶段可邀请用户参与。

在进行概念设计时, 可先设计各个应用的视图, 即各个应用所看到的数据及其结构, 然后再进行视图集成, 以形成一个单一的概念数据模型。

4. 逻辑设计

逻辑设计也称为逻辑结构设计, 其任务是将概念模型转化为某个特定的 DBMS 上的逻辑模型。设计逻辑结构时, 首先为概念模型选定一个合适的逻辑模型 (例如, 关系模型、网状模型或层次模型), 然后将其转化为由特定 DBMS 支持的逻辑模型, 最后对逻辑模型进行优化。

逻辑设计的目的是将概念设计阶段设计好的 E-R 图转换为与选用的具体机器上的 DBMS 所支持的数据模型相符合的逻辑结构。

5. 物理设计

物理设计也称为物理结构设计, 其任务是对给定的逻辑模型选取一个最适合应用环境的物

理结构，所谓数据库的物理结构，主要是指数据库在物理设备上的存储结构和存取方法。物理设计的步骤如下：

- (1) 设计存储记录结构，包括记录的组成、数据项的类型和长度，以及逻辑记录到存储记录的映射。
- (2) 确定数据存储安排。
- (3) 设计访问方法，为存储在物理设备上的数据提供存储和检索的能力。
- (4) 进行完整性和安全性的分析与设计。
- (5) 数据库程序设计。

5.4.2 实体联系模型

E-R 模型也称为 E-R 图，它是描述概念世界、建立概念模型的实用工具。在 E-R 图中，主要包括以下三个要素：

- (1) 实体（型）。实体用矩形框表示，框内标注实体名称。
- (2) 属性。单值属性用椭圆形表示，并用连线与实体连接起来；如果是多值属性，在椭圆虚线外面再套实线椭圆；如果是派生属性，则用虚线椭圆表示。其中，多值属性可以有一个或两个以上的值，例如，学员信息数据库中可能包含关于他们个人兴趣的数据，一个学员可能有运动、电影、投资和烹饪等多个兴趣；派生属性是从基本属性计算出来的属性，例如，学员的总成绩和平均成绩等。
- (3) 实体之间的联系。实体之间的联系用菱形框表示，框内标注联系名称，并用连线将菱形框分别与有关实体相连，并在连线上注明联系类型。

例如，图 5-4 就是某在线教育平台系统的一个 E-R 图（为了简单起见，省略了部分实体的属性和联系的属性）。

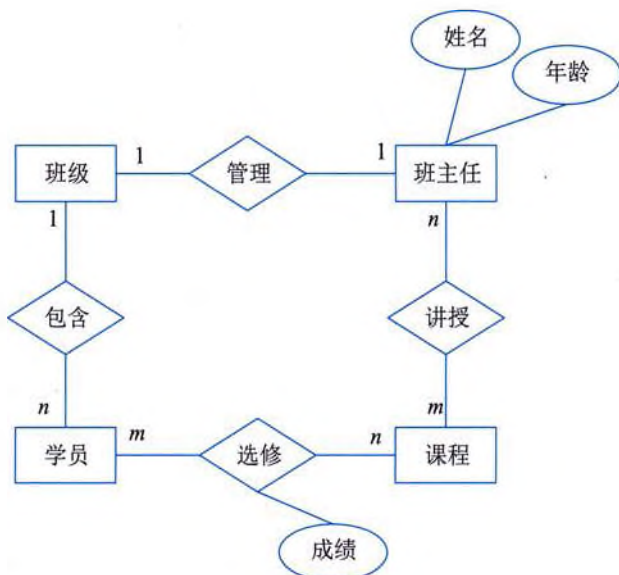


图 5-4 某在线教育平台系统的 E-R 图

1. 联系的类型

E-R 图中的联系可以归结为三种类型, 分别是一对一联系、一对多联系和多对多联系。

(1) 一对一联系 ($1:1$)。设 A, B 为两个实体集, 若 A 中的每个实体至多和 B 中的一个实体有联系, 反过来, B 中的每个实体至多和 A 中的一个实体有联系, 则称 A 对 B 或 B 对 A 是 $1:1$ 联系。要注意的是, $1:1$ 联系不一定都是一一对应的关系, 可能存在着无对应。例如, 在图 5-4 中, 一个班只有一个班主任, 一个辅导老师不能同时兼任两个班的班主任, 由于辅导老师紧缺, 某个班的班主任也可能暂缺。

(2) 一对多联系 ($1:n$)。如果实体集 A 中的每个实体可以和 B 中的几个实体有联系, 而 B 中的每个实体至少和 A 中的一个实体有联系, 则 A 对 B 属于 $1:n$ 联系。例如, 在图 5-4 中, 一个班级有多个学员, 而一个学员只能编排在一个班级, 班级与学员之间的关系属于一对多联系。

(3) 多对多联系 ($m:n$)。若实体集 A 中的每个实体可与 B 中的多个实体有联系, 反过来, B 中的每个实体也可以与 A 中的多个实体有联系, 则称 A 对 B 或 B 对 A 是 $m:n$ 联系。例如, 在图 5-4 中, 一个学员可以选修多门课程, 一门课程由多个学员选修, 学员和课程之间存在多对多联系。

2. E-R 图的集成

在数据库的概念设计过程中, 先设计各子系统的局部 E-R 图, 其设计过程是: 首先, 确定局部视图的范围; 然后, 识别实体及其标识, 确定实体之间的联系; 最后, 分配实体及联系的属性。各子系统的局部 E-R 图设计好后, 下一步就是要将所有的分 E-R 图综合成一个系统的总体 E-R 图, 一般称为视图的集成。视图集成通常有两种方式: 一种方式是多个局部 E-R 图一次集成, 这种方式比较复杂, 做起来难度较大; 另一种方式是逐步集成, 用累加的方式一次集成两个局部 E-R 图, 这种方式每次只集成两个局部 E-R 图, 可以降低复杂度。

由于各子系统应用所面临的问题不同, 且通常是由不同的设计人员进行局部视图设计, 这就导致各个局部 E-R 图之间必定会存在许多不一致的问题, 称之为冲突。因此, 在合并 E-R 图时, 不能简单地将各个局部 E-R 图画到一起, 而是必须着力消除各个局部 E-R 图中的不一致, 以形成一个能为全系统中所有用户共同理解和接受的统一的概念模型。各局部 E-R 图之间的冲突主要有三类: 属性冲突、命名冲突和结构冲突。

(1) 属性冲突。属性冲突包括属性域冲突和属性取值冲突。属性冲突理论上好解决, 只要换成相同的属性就可以了, 但实际上需要各部门协商, 解决起来并不简单。

(2) 命名冲突。命名冲突包括同名异义和异名同义。处理命名冲突通常也像处理属性冲突一样, 通过讨论和协商等行政手段加以解决。

(3) 结构冲突。结构冲突包括同一对象在不同应用中具有不同的抽象, 以及同一实体在不同局部 E-R 图所包含的属性个数和属性排列次序不完全相同。对于前者, 解决办法是将属性变换为实体或将实体变换为属性, 使同一对象具有相同的抽象。对于后者, 解决办法是使该实体的属性取各局部 E-R 图中属性的并集, 再适当调整属性的次序。

另外, 实体间的联系在不同的局部 E-R 图中可能为不同的类型, 其解决方法是根据应用的语义对实体联系的类型进行综合或调整。在初步的 E-R 图中, 可能存在一些冗余的数据和实体间冗余

的联系。冗余数据和冗余联系容易破坏数据库的完整性，给数据库维护增加困难，应当予以消除。

3. E-R 图向关系模式的转换

E-R 图向关系模式的转换属于数据库逻辑设计阶段的工作，该阶段需要将 E-R 模型转换为某种 DBMS 能处理的关系模式，具体转换规则如下：

(1) 一个实体转换为一个关系模式，实体的属性就是关系的属性，实体的主键就是关系的主键。

(2) 一个 1 : 1 联系可以转换为一个独立的关系模式，也可以与任意一端对应的关系模式合并。如果转换为一个独立的关系模式，则与该联系相连的各实体的主键和联系本身的属性均转换为关系的属性，每个实体的主键均是该关系的键属性；如果与某一端实体对应的关系模式合并，则需要在该关系模式的属性中加入另一个关系模式的主键和联系本身的属性。

(3) 一个 1 : n 联系可以转换为一个独立的关系模式，也可以与任意 n 端对应的关系模式合并。如果转换为一个独立的关系模式，则与该联系相连的各实体的主键和联系本身的属性均转换为关系的属性，而关系的主键为 n 端实体的主键；如果与 n 端实体对应的关系模式合并，则需要在该关系模式的属性中加入 1 端关系模式的主键和联系本身的属性。

(4) 一个 $m : n$ 联系转换为一个独立的关系模式，与该联系相连的各实体的主键以及联系本身的属性均转换为关系的属性，而关系的主键为各实体主键的组合。

(5) 三个以上实体间的一个多元联系可以转换为一个独立的关系模式，与该联系相连的各实体的主键和联系本身的属性均转换为关系的属性，而关系的主键为各实体主键的组合。

另外，还有 4 种情况是需要特别注意的：

(1) 多值属性的处理。如果 E-R 图中某实体具有一个多值属性，则应该进行优化，将该属性提升为一个实体，通常称为弱实体；或者在转化为关系模式时，将实体的主键与多值属性单独构成一个关系模式。

(2) BLOB 型属性的处理。典型的 BLOB 是一张图片或一个声音文件，由于它们的容量比较大，必须使用特殊的方式来处理。处理 BLOB 的主要思想就是让文件处理器（例如，数据库管理器）不去理会文件是什么，而是关心如何去处理它。因此，从优化的角度考虑，应采用的设计方案是将 BLOB 属性与关系的主键独立为一个关系模式。

(3) 派生属性的处理。因为派生属性可由其他属性计算得到，因此，在转化成关系模式时，通常不转换派生属性。

(4) 在面向对象的模型中，本节的关系模式就对应类，关系模式的属性就对应类的属性。

5.5 分布式数据库系统

分布式数据库系统是数据库技术与网络技术相结合的产物，其基本思想是将传统的集中式数据库中的数据分布于网络上的多台计算机中。分布式数据库系统通常使用较小的计算机系统，每台计算机可单独放在一个地方，每台计算机中都有 DBMS 的一份完整的复制副本，并具有自己局部的数据库，位于不同地点的许多计算机通过网络互相连接，共同组成一个完整的、全局的大型数据库。

5.5.1 分布式数据库概述

分布式数据库是由一组数据组成的, 这组数据分布在计算机网络的不同计算机上, 网络中的每个节点具有独立处理的能力(称为场地自治), 它可以执行局部应用, 同时, 每个节点也能通过网络通信子系统执行全局应用。分布式数据库系统是在集中式数据库系统技术的基础上发展起来的, 具有如下特点:

(1) 数据独立性。在分布式数据库系统中, 数据独立性这一特性更加重要, 并具有更多的内容。除了数据的逻辑独立性与物理独立性外, 还有数据分布独立性(分布透明性)。

(2) 集中与自治共享结合的控制结构。各局部的 DBMS 可以独立地管理局部数据库, 具有自治的功能。同时, 系统又设有集中控制机制, 协调各局部 DBMS 的工作, 执行全局应用。

(3) 适当增加数据冗余度。在不同的场地存储同一数据的多个副本, 这样可以提高系统的可靠性和可用性, 同时也能提高系统性能。

(4) 全局的一致性、可串行性和可恢复性。

1. 分布式数据库的体系结构

分布式数据库的体系结构如图 5-5 所示。在分布式数据库中, 局部 DBMS 中的内模式和概念模式与集中数据库中是完全一致的, 不同之处在于新增的全局 DBMS, 而整个全局 DBMS 可以看作是相对于局部概念模式的外模式。

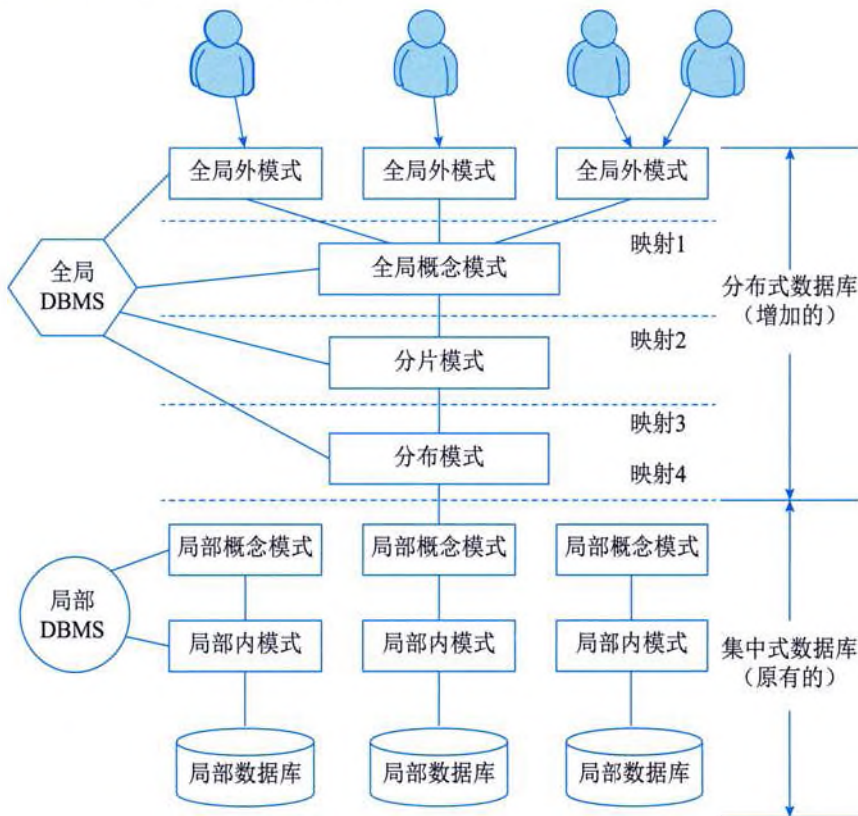


图 5-5 分布式数据库的体系结构

外模式部分由一系列的分布模式、分片模式、全局概念模式和全局外模式以及多级映射组成，这样用户在使用分布式数据库时，可以使用与集中式数据库相同的方式。

(1) 全局外模式。全局外模式是全局应用的用户视图，是全局概念模式的子集，该层直接与用户（或应用程序）交互。

(2) 全局概念模式。全局概念模式定义分布式数据库中数据的整体逻辑结构，数据就如同根本没有分布一样，可用传统的集中式数据库中所采用的方法进行定义。全局概念模式中所用的数据模型应该易于向其他层次的模式映射，通常采用关系模型。

(3) 分片模式。在某些情况下，需要将一个关系模式分解成几个数据片，分片模式正是用于完成此项工作的。有关数据分片的详细内容将在 5.5.2 小节中介绍。

(4) 分布模式。分布式数据库的本质特性就是数据分布在不同的物理位置。分布模式的主要职责是定义数据片段（即分片模式的处理结果）的存放节点。分布模式的映射类型决定了分布式数据库是冗余的还是非冗余的。若映射是一对多的，即一个片段分配到多个节点上存放，则是冗余的分布式数据库，否则是不冗余的分布式数据库。根据分布模式提供的信息，一个全局查询可分解为若干个子查询，每个子查询要访问的数据属于同一场地的局部数据库。由分布模式到各局部数据库的映射（图 5-5 中的映射 4）将存储在局部场地的全局关系（或全局关系的片段）映射为各局部概念模式，采用局部场地的 DBMS 所支持的数据模型。

(5) 局部概念模式。局部概念模式是局部数据库的概念模式。

(6) 局部内模式。局部内模式是局部数据库的内模式。

虽然从理论上来说，分布式数据库的模式结构有图 5-5 所示的 6 个层次，但实际上，并非所有分布式数据库都具有这种结构。

2. 分布式数据库的优点

分布式数据库的物理层面分布、逻辑层面统一的特色，让它具有一些集中式数据库所不可及的优点：

(1) 分布式数据库可以解决企业部门分散而数据需要相互联系的问题。例如，就银行系统而言，总行与各分行处于不同的城市或城市中的不同地区，在业务上它们需要处理各自的数据，也需要彼此之间的交换和处理，这就需要分布式数据库系统。

(2) 如果企业需要增加新的相对自主的部门来扩充机构，则分布式数据库系统可以在对当前机构影响最小的情况下进行扩充。

(3) 分布式数据库可以满足均衡负载的需要。数据的分片使局部应用达到最大，这使得各服务器之间的相互干扰降到最低。负载在各服务器之间分担，可以避免临界瓶颈。

(4) 当企业已存在几个数据库系统，而且实现全局应用的必要性增加时，就可以由这些数据库自下而上构成分布式数据库系统。

(5) 相等规模的分布式数据库系统在出现故障的概率上不会比集中式数据库系统低，但由于其故障的影响仅限于局部数据应用，因此，就整个系统来说，它的可靠性是比较高的。

5.5.2 数据分片

数据分片将数据库整体逻辑结构分解为合适的逻辑单位（片段），然后由分布模式来定义

片段及其副本在各场地的物理分布,其主要目的是提高访问的局部性,有利于按照用户的需求,组织数据的分布和控制数据的冗余度。

分片的方式有多种,水平分片和垂直分片是两种基本的分片方式,混合分片和导出分片是较复杂的分片方式。

(1) 水平分片。水平分片将一个全局关系中的元组分裂成多个子集,每个子集为一个片段。分片条件由关系中的属性值表示。对于水平分片,重构全局关系可通过关系的并操作实现。

(2) 垂直分片。垂直分片将一个全局关系按属性分裂成多个子集,应满足不相交性(关键字除外)。对于垂直分片,重构全局关系可通过连接运算实现。

(3) 导出分片。导出分片又称为导出水平分片,即水平分片的条件不是本关系属性的条件,而是其他关系属性的条件。

(4) 混合分片。混合分片是在分片中采用水平分片和垂直分片两种形式的混合。

5.6 数据仓库技术

数据仓库是一个面向主题的、集成的、相对稳定的、反映历史变化的数据集合,用于支持管理决策。近年来,人们对数据仓库技术的关注程度越来越高,其原因是过去的几十年中,建设了无数的应用系统,积累了大量的数据,但这些数据没有得到很好的利用,有时反而成为企业的负担。如图 5-6 所示为数据仓库的体系结构。

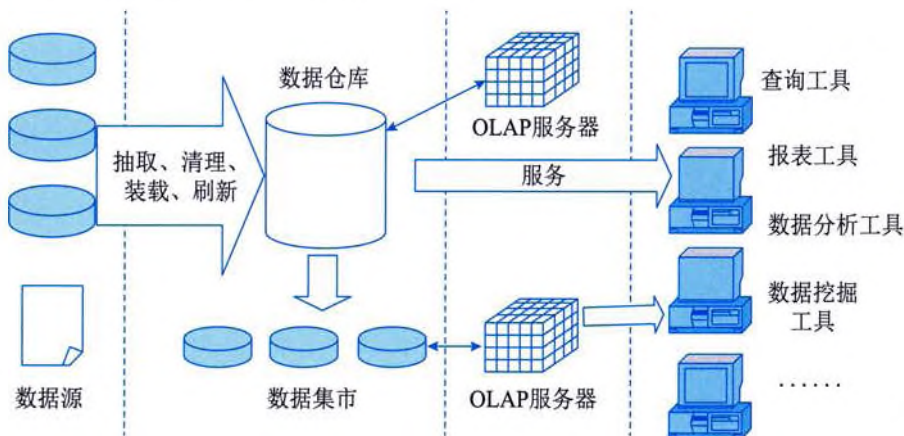


图 5-6 数据仓库的体系结构

在图 5-6 中,数据源是数据仓库系统的基础,是整个系统的数据源泉。OLAP (On-Line Analytical Processing, 联机分析处理) 服务器对分析需要的数据进行有效集成,按多维模型予以组织,以便进行多角度、多层次的分析,并发现趋势。前端工具主要包括各种报表工具、查询工具、数据分析工具和数据挖掘工具,以及各种基于数据仓库或数据集市的应用开发工具。其中,数据分析工具主要针对 OLAP 服务器,报表工具、数据挖掘工具主要针对数据仓库。

5.6.1 联机分析处理

数据处理大致可以分成两大类，分别是联机事务处理（On-Line Transaction Processing, OLTP）和联机分析处理（OLAP）。OLTP 是传统数据库的主要应用，支持基本的、日常的事务处理；OLAP 是数据仓库系统的主要应用，支持复杂的分析操作，侧重决策支持，并且提供直观易懂的查询结果。表 5-12 列出了 OLTP 与 OLAP 之间的比较。

表 5-12 OLTP 与 OLAP 之间的比较

项 目	OLTP	OLAP
用户	操作人员，低层管理人员	决策人员，高层管理人员
功能	日常操作处理	分析决策
DB 设计	面向应用	面向主题
数据	当前的、最新的、细节的、二维的、分立的	历史的、聚集的、多维的、集成的、统一的
存取	读 / 写数十条记录	读上百万条记录
工作单位	简单的事务	复杂的查询
用户数	多	少
DB 大小	MB 或 GB 级	GB 或 TB 级

从表 5-12 中可以看出，在 OLTP 中，数据是以二维表的形式来进行组织的，但在 OLAP 中，数据通常是多维的。这里的“维”是人们观察客观世界的角度，是一种高层次的类型划分。“维”一般包含层次关系，这种层次关系有时会相当复杂。通过将一个实体的多项重要的属性定义为多个维，使用户能对不同维上的数据进行比较。因此，OLAP 也可以说是多维数据分析工具的集合。

1. 数据立方体

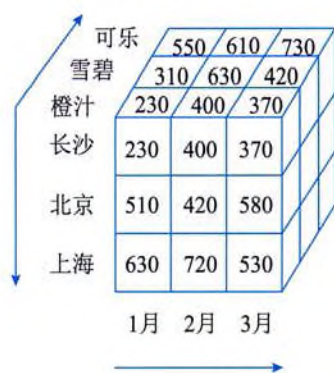


图 5-7 数据立方体示意图

在多维的数据结构中，三维结构（Data Cube，数据立方体）最为直观，如图 5-7 所示，该立方体中包含了一个连锁超市在各地区不同月份的各种饮料的销售情况。

通过图 5-7 可以发现，其实绝大多数的应用，只需要用到立方体的一部分。例如，需要了解长沙地区的饮料销售情况，则只要取立方体的最上面的一片数据即可，若需要了解长沙地区 3 月份的饮料销售情况，则只要取立方体最上面一层的最右边一列数据即可。这就涉及数据立方体的相应操作。

2. 多维分析

OLAP 的基本多维分析操作有钻取、切片和切块、旋转等。

（1）钻取（Drill）。钻取是改变维的层次，变换分析的粒度。它包括向上钻取（Drill Up）和向下钻取（Drill Down）。向上钻取是在某一维上将低层次的细节数据概括到高层次的汇总数据，或者减少维数，是一种自动生成汇总行的分析方法。通过向导的方式，用户可以定义和分析因素的汇总行。例如，要了解各地区各月份的饮料销售情况，可以生成地区和（或）月份的

合计行。而向下钻取则相反, 它从汇总数据深入到细节数据进行观察或增加新的维。例如, 用户分析各地区的销售情况时, 可以对某个城市的销售额细分为各个月份的销售额。通过钻取功能, 用户对数据能更深入地了解, 更容易发现问题, 做出正确的决策。

(2) 切片和切块 (Slice and Dice)。切片和切块是在一部分维上选定值后, 关心度量数据在剩余维上的分布。如果剩余的维只有两个, 则是切片; 如果剩余的维有三个以上, 则是切块。例如, 如果需要在图 5-7 中获取按月的销售数据, 则属于数据切片, 因为数据立方体按月份分成了数据片, 形成了二维结构。但如果需要获取的是按地区的数据, 例如, 华东区 (包括上海、南京、杭州等), 华北区 (包括北京、天津等), 则属于数据分块, 因为得到的数据是三维结构的。

(3) 旋转 (Pivot) 是变换维的方向, 即重新安排维的放置 (例如, 行列互换等)。

3. 实现方法

OLAP 有多种实现方法, 根据存储数据的方式不同, 可以分为关系型 OLAP (Relational OLAP, ROLAP)、多维型 OLAP (Multidimensional OLAP, MOLAP) 和混合型 OLAP (Hybrid OLAP, HOLAP)。

ROLAP 表示基于关系数据库的 OLAP 实现。以关系数据库为核心, 以关系型结构进行多维数据的表示和存储。ROLAP 将多维数据库的多维结构划分为两类表: 一类是事实表, 用来存储数据和维关键字; 另一类是维表, 即对每个维至少使用一个表来存放维的层次和成员类别等维的描述信息。维表和事实表通过主键和外键联系在一起, 形成“星形模式”。对于层次复杂的维, 为避免冗余数据占用过大的存储空间, 可以使用多个表来描述, 这种星形模式的扩展称为“雪花模式”。

MOLAP 表示基于多维数据组织的 OLAP 实现, 以多维数据组织方式为核心。也就是说, MOLAP 使用多维数组存储数据。如果是三维数据, 则其在存储中将形成数据立方体结构。

HOLAP 表示基于混合数据组织的 OLAP 实现, 例如, 低层是关系型的, 高层是多维型的, 这种方式具有更好的灵活性。

5.6.2 数据仓库概述

企业数据仓库的建设, 是以现有企业业务系统和大量业务数据的积累为基础的。数据仓库不是静态的概念, 只有将信息及时交给需要这些信息的使用者, 供他们做出改善其业务经营的决策, 信息才能发挥作用。而将信息加以整理归纳和重组, 并及时提供给相应的管理决策人员, 是数据仓库的根本任务。因此, 从产业界的角度看, 数据仓库建设是一个工程, 是一个过程。

1. ETL 过程

针对现有各业务系统的数据, 进行抽取、清理, 并有效集成, 按照主题进行组织, 整个过程可以简称为抽取、转换和加载 (Extraction-Transformation-Loading, ETL) 过程。ETL 负责将分布的、异构数据源中的数据 (例如, 关系数据、平面数据文件等) 抽取到临时中间层后进行清洗、转换和集成, 最后加载到数据仓库或数据集中, 成为 OLAP 和数据挖掘的基础。

数据仓库是一个独立的数据环境，需要通过抽取过程将数据从 OLTP 环境、外部数据源和脱机的数据存储介质中导入数据仓库。在技术上，ETL 主要涉及关联、转换、增量、调度和监控等几个方面。数据仓库系统中的数据不要求与 OLTP 系统中的数据实时同步，因此，ETL 可以定时进行。但多个 ETL 的操作时间、顺序和成败对数据仓库中信息的有效性至关重要。

2. 数据仓库的分类

从结构的角度看，有三种数据仓库模型，分别是企业仓库（Enterprise Warehouse）、数据集市（Datamart）和虚拟仓库（Virtual Warehouse）。

（1）企业仓库。企业仓库面向企业级应用，它搜集了企业的各个主题的所有信息，提供全企业范围的数据集成，数据通常来自多个操作型数据库和外部信息提供者，并且是跨多个功能范围的。企业仓库通常包含详细数据和汇总数据，数据量可达 TB 级。

（2）数据集市。数据集市面向企业部门级应用，它包含对特定用户有用的、企业范围数据的一个子集，它的范围限定在选定的主题中。根据数据来源不同，数据集市可以分为两种，分别是独立数据集市（Independent Datamart）和从属数据集市（Dependent Datamart）。

从属数据集市的数据直接来自于中央数据仓库，有利于保持数据的一致性，因为来自同一数据源，并且已经经过一致性处理和检验。从属数据集市的作用在于，为一些部门建立数据集市，将需要的数据复制、加工到其中，这样不仅可以提高部门的访问速度，也能满足部门的一些特殊的分析需求。

独立数据集市的数据直接来自于业务系统，由于为各个部门都建立了各自的数据集市，而当需要从整体上建立数据仓库时，由于各部门的特殊需要不同，不同数据集市中的数据表达也有所不同，将这种不一致的数据整合到一个中心数据仓库时，可能会遇到一些困难，例如，重新设计和各部门协调等。独立数据集市的优点是建立迅速、价格相对低廉。建立独立数据集市的原因往往是出于投资方面的考虑或工期的紧迫，或解决某部门的迫切需要。然而，需要注意的是，在设计其他部门的数据集市或中心数据仓库时，要充分考虑现有数据集市的设计，以避免由于设计的不一致性而造成后期整合的困难及昂贵的费用。

（3）虚拟仓库。虚拟仓库是操作型数据库上视图的集合。为了有效地处理查询，只有一些可能的汇总视图被物化。虚拟仓库易于建立，但需要操作型数据库服务器具有剩余的工作能力。

3. 非结构化数据与数据仓库

图 5-6 展示了数据仓库的体系结构，其中，第一步便是从多个数据源获取数据。这些数据源不仅限于企业内部的应用系统数据库。因为对于一个用于决策分析的系统而言，仅有企业内部数据是不够的。合理地采用外部数据（例如，报纸、期刊、电视等媒体的报道，一些商业机构的调查报告）能使分析和决策更为准确。而这些外部数据通常都是非结构化的数据。因此，如何用数据仓库管理非结构化数据，也是数据仓库应用中的一个重要问题。

为了更好地管理非结构化数据，数据仓库采用了元数据（当然，元数据不仅用于此），元数据可用于记录数据的文件标识符、进入数据仓库的日期、文件描述、文件来源、文件源的取得日期、文件的分类、索引字、清理日期、物理地址引用、文件长度和相关参考等信息。因此，管理人员可以通过元数据来获得非结构化数据的信息，在许多情况下，管理人员甚至不用看源

文件, 只要看元数据即可。在清除不相关的或过时的文件时, 浏览元数据可为管理人员减少大量的工作。

5.6.3 数据仓库的设计方法

数据仓库的设计方法通常有三种, 分别是自顶向下的方法、自底向上的方法, 以及两者结合的混合方法。

1. 自顶向下的方法

自顶向下的方法由总体规划和设计开始, 在对原始数据进行抽取、转换和迁移等处理之后, 将数据输出至一个集中的数据驻留单元。然后, 数据和元数据装载进入数据仓库。这样建立起来的数据仓库是企业级的, 当建立好数据仓库以后, 各个部门再从数据仓库中获取部门所需要的数据, 形成数据集市, 即从属数据集市。

自顶向下的方法在实际应用中会遇到很多难题, 例如:

- (1) 投资大。因为建立的数据仓库是面向企业的, 涉及面广。
- (2) 应用周期较长。因为项目较大, 开发周期相对较长, 应用时间也长。
- (3) 需求难以确定。因为建立数据仓库的主要原因是利用其进行决策分析, 这种功能在企业战略的应用范围中通常是很难确定的, 数据仓库的应用往往超出企业当前的实际业务范围。

2. 自底向上的方法

自底向上的方法的核心思想是从企业中最关键的部门(或功能需求)开始, 先以最少的投资完成当前的需求, 获得最快的回报, 然后再不断扩充、不断完善。以该方法进行数据仓库的设计, 最先产生的是独立数据集市, 而后从多个独立数据集中抽取数据, 形成企业级的数据仓库。

自底向上的方法的优点在于企业能够以较小的投入获得较高的数据仓库应用收益。在开发过程中, 人员投入较少, 也容易获得成效。当然, 如果某个项目的开发失败, 可能造成企业整个数据仓库系统开发的延迟。该方法一般用于企业希望对数据仓库技术进行评价, 以确定该技术的应用方式、地点和时间, 或希望了解实现和运行数据仓库所需要的各种费用, 或在数据仓库的应用目标并不是很明确时使用。

3. 混合法

混合法是自顶向下和自底向上的方法联合使用, 具有两种方法的优点, 既能快速地完成数据仓库的开发与应用, 还可建立具有长远价值的数据仓库方案。

5.7 数据挖掘技术

数据挖掘是采用数学的、统计的、人工智能和神经网络等领域的科学方法, 从大量数据中挖掘出隐含的、先前未知的、对决策有潜在价值的关系、模式和趋势, 并用这些知识和规则建立用于决策支持的模型, 为商业智能系统服务的各业务领域提供预测性决策支持的方法、工具和过程。

5.7.1 数据挖掘概述

数据挖掘与传统的数据分析（例如，查询、报表、联机应用分析）的本质区别是数据挖掘是在没有明确假设的前提下去挖掘信息、发现知识。数据挖掘所得到的信息应具有先前未知、有效和可实用三个特征。先前未知的信息是指该信息是预先未曾预料到的，即数据挖掘是要发现那些不能靠直觉发现的信息或知识，甚至是违背直觉的信息或知识，挖掘出的信息越是出乎意料，就可能越有价值。

1. 数据挖掘的体系结构

如图 5-8 所示为数据挖掘的体系结构，它展示了数据挖掘的流程，说明了数据挖掘是怎样找到新规律的。

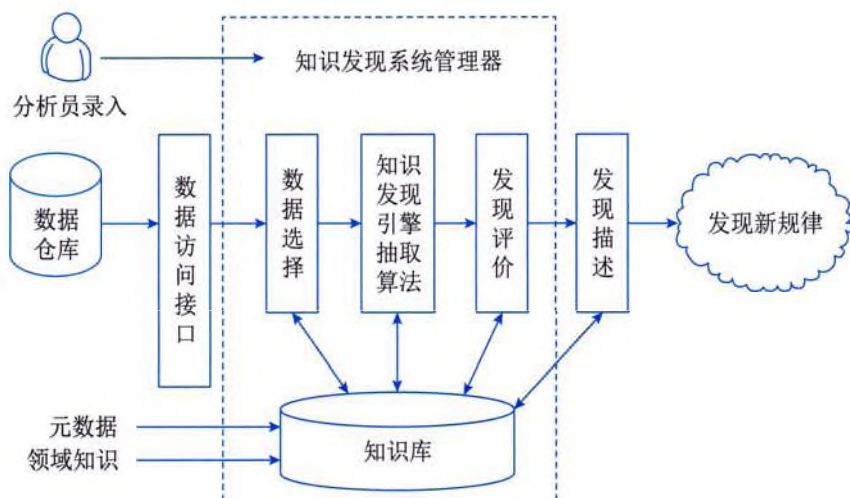


图 5-8 数据挖掘的体系结构

(1) 知识发现系统管理器。控制并管理知识发现过程，录入知识库中的信息用于驱动数据选择过程、抽取算法选择，以及使用过程和发现评价过程。

(2) 知识库。知识库包含源于多方面的必需的信息，可以将元数据输入数据仓库中，以描述数据仓库的数据结构，输入关键数据属性、规则和数据层次等。

(3) 数据访问接口。知识发现系统利用数据库的查询机制从数据仓库中提取数据，可使用 SQL 查询语言，结合知识库中的数据仓库元数据，指导从数据仓库中提取需要的数据。

(4) 数据选择。确定从数据仓库中需要抽取的数据及其结构。

(5) 知识发现引擎。将知识库中的抽取算法提供给抽取的数据，目的是要抽取数据元素间的模式和关系。

(6) 发现评价。分析员要寻找关注性的数据模式，选出那些关注性信息。

(7) 发现描述。发现描述部分提供两种功能：一种是以发现评价辅助分析员在知识库中保存所发现的信息，以备将来引用和使用；另一种是发现描述为决策提供辅助决策。

2. 数据挖掘的流程

数据挖掘是一个完整的过程,该过程从大型数据库中挖掘先前未知的、有效的、可实用的信息,并使用这些信息做出决策或丰富知识。数据挖掘的流程大致如下:

(1) 问题定义。在开始数据挖掘之前最先也是最重要的要求就是熟悉背景知识,弄清用户的需求。缺少背景知识就不能明确定义要解决的问题,也就不能为挖掘准备优质的数据,也很难正确地解释得到的结果。要想充分发挥数据挖掘的价值,必须要对目标有清晰明确的定义,即决定到底想干什么。

(2) 建立数据挖掘库。要进行数据挖掘必须收集要挖掘的数据资源,一般需要将要挖掘的数据都收集到一个数据库中,而不是采用原有的数据库或数据仓库。这是因为大部分情况下需要修改要挖掘的数据,而且还会遇到采用外部数据的情况;另外,数据挖掘还要对数据进行各种纷繁复杂的统计分析,而数据仓库可能不支持这些数据结构。

(3) 分析数据。分析数据是对数据深入调查的过程。从数据集中找出规律和趋势,用聚类分析区分类别,理清多因素相互影响的、十分复杂的关系,发现诸因素之间的相关性。

(4) 调整数据。通过上述步骤的操作,对数据的状态和趋势有了进一步的了解,这时,要尽可能对问题解决的要求做进一步明确化和量化。针对问题的需求对数据进行增删,按照对整个数据挖掘过程的新认识组合或生成一个新的变量,以体现对状态的有效描述。

(5) 模型化。在问题进一步明确、数据结构和内容进一步调整的基础上,就可以建立形成知识的模型。这一步是数据挖掘的核心环节,一般运用神经网络、决策树和数理统计等方法来建立模型,有关这些技术与方法的详细内容将在 5.7.2 小节中介绍。

(6) 评价和解释。所得到的模型有可能是没有实际意义或没有实用价值的,也有可能不能准确反映数据的真实意义,甚至在某些情况下是与事实相反的。因此,需要评估和确定哪些是有效的、有用的模式。评估的方法既可以是直接使用原先建立的挖掘数据库中的数据来进行检验,也可以另外拿一批数据对其进行检验,还可以在实际运行的环境中取出新数据进行检验。

数据挖掘是一个多种专家合作的过程,也是一个在资金上和技术上高投入的过程。这一过程要反复进行,在此过程中不断地趋近事物的本质,不断地优选问题的解决方案。

5.7.2 常用技术与方法

从技术上来看,数据挖掘就是从大量的、不完全的、有噪声的、模糊的、随机的实际应用中,提取隐含在其中的、人们事先不知道的、但又是潜在有用的信息和知识的过程。这个定义包括几层含义:数据源必须是真实的、大量的、含噪声的;发现的是用户感兴趣的知识;发现的知识要可接受、可理解、可运用;并不要求发现放之四海而皆准的知识,仅支持特定地发现问题即可。

从商业角度来看,数据挖掘是一种新的商业信息处理技术,其主要特点是对商业数据库中的大量业务数据进行抽取、转换、分析和其他模型化处理,从中提取辅助商业决策的关键性数据。

1. 数据挖掘的常用技术

数据挖掘中的关键技术是进行模式和关系识别的算法。下面介绍几种数据挖掘和知识发现的技术，它们分别从不同的角度进行数据挖掘和知识发现。

(1) 决策树方法。决策树方法利用信息论中的互信息（信息增益）寻找数据库中具有最大信息量的属性，建立决策树的一个结点，再根据属性的不同取值建立树的分支。

(2) 分类方法。分类方法将数据按照含义划分成组，可用该方法生成感兴趣的侧面，可用于自动发现类，例如，模式识别、侧面生成、线性聚簇和概念聚簇等。

(3) 粗糙集（Rough Set）方法。粗糙集的研究主要基于分类。分类和概念（Concept）同义，一种类别对应于一个概念。知识由概念组成，如果某知识中含有不精确概念，则该知识就不精确，粗糙集通过上近似概念和下近似概念来表示不精确概念。

(4) 神经网络。神经网络通过学习待分析数据中的模式来构造模型，它可对隐式类型进行分析，适用于对非线性的、复杂的或高噪声的数据进行建模。神经网络技术模拟人脑神经元结构，由神经元互连或按层组织的结点构成。通常，神经模型由三个层次组成，分别是输入层、中间层和输出层。每个神经元求得输入值，再计算总输入值，由过滤机制（例如，阈值）比较总输入，然后确定它自己的输出值。

(5) 关联规则。关联规则是指搜索业务系统中的所有细节和事务，从中找出重复出现概率很高的模式，它以大的事务数据库为基础，其中每个事务都被定义为一组相关数据项。用关联找出所有能将一组事件（或数据项）与另一组事件（或数据项）联系起来的规则。关联算法的目的是成为 SQL 的扩充，这种算法可以通过规范的查询技术应用于受限的关系数据集。

(6) 概念树方法。对数据库中记录的属性按归类方式进行抽象，建立起来的层次结构称为概念树。例如，某在线教育平台系统中的“课程”概念树的最下层是具体课程名称（例如，数据结构、系统分析与设计等），它的直接上层是课程类别（例如，软件类、网络类等），课程类别的直接上层是专业类别（例如，计算机科学与技术、通信工程等）。利用概念树提升的方法可以大大浓缩数据库中的记录。对多个属性的概念树进行提升，将得到高度概括的知识基表，然后再将它转换成规则。

(7) 遗传算法。遗传算法是模拟生物进化过程的算法，由繁殖、交叉和变异三个基本算子组成。繁殖也称为选择，是从一个旧种群（父代）选出生命力强的个体产生新种群（后代）的过程；交叉也称为重组，是指选择两个不同个体（染色体）的部分（基因）进行交换，形成新个体；变异也称为突变，是指对某些个体的某些基因进行变异（1 变 0、0 变 1）。遗传算法可起到产生优良后代的作用，这些后代需满足适应值，经过若干代的遗传，将得到满足要求的后代（问题的解）。

(8) 依赖性分析。依赖性分析是指在数据仓库的条目或对象之间抽取依赖性，它展示了数据之间未知的依赖关系，依赖性是一个带有置信度因子的可能值。可以用依赖性分析方法从某个数据对象的信息来推断另一个数据对象的信息。

(9) 公式发现。在工程和科学数据库中，对若干数据项（变量）进行一定的数学运算，求得相应的数学公式。其基本思想是，对数据项进行初等数学运算，形成组合数据项，若它的值为常数项，就得到了组合数据项等于常数的公式。

(10) 统计分析方法。在数据库属性之间通常存在两种关系, 分别是函数关系(能用某个函数表示的确定性关系)和相关关系(不能用函数表示的确定性关系)。对这些关系的分析可以采用回归分析、相关分析或主成分分析等统计分析方法。

(11) 模糊论方法。利用模糊集合理论对实际问题进行模糊评判、模糊决策、模糊模式识别和模糊聚类分析。模糊性是客观存在的, 系统的复杂性越高, 精确化能力就越低, 即模糊性越强。

(12) 可视化分析。可视化分析可给出带有多变量的图形化分析数据, 帮助用户进行分析。可视化数据分析技术拓宽了传统的图表功能, 使用户对数据的剖析更清楚。例如, 将数据库中的多维数据变成多种图形, 这对揭示数据的状况、内在本质及规律性起到了很重要的作用。

2. 数据挖掘的分析方法

从功能上可以将数据挖掘的分析方法划分为6种, 即关联分析、序列分析、分类分析、聚类分析、预测分析和时间序列分析。

(1) 关联分析。关联分析主要用于发现不同事件之间的关联性, 即一个事件发生的同时, 另一个事件也经常发生。关联分析的重点在于快速发现那些有实用价值的关联发生的事件。其主要依据是事件发生的概率和条件概率应该符合一定的统计意义。在进行关联分析的同时, 还需要计算两个参数, 分别是最小置信度(可信度)和最小支持度。前者表示规则需满足的最低可靠度, 用以过滤掉可能性过小的规则; 后者则用来表示规则在统计意义上需满足的最低程度。

(2) 序列分析。序列分析主要用于发现一定时间间隔内接连发生的事件, 这些事件构成一个序列, 发现的序列应该具有普遍意义, 其依据除了统计上的概率之外, 还要加上时间的约束。在进行序列分析时, 也应计算置信度和支持度。

(3) 分类分析。分类分析通过分析具有类别的样本特点, 得到决定样本属于各种类别的规则或方法。利用这些规则和方法对未知类别的样本分类时应该具有一定的准确度。其主要方法有基于统计学的贝叶斯方法、神经网络方法、决策树方法等。分类分析时首先为每个记录赋予一个标记(一组具有不同特征类别), 即按标记分类记录, 然后检查这些标定的记录, 描述出这些记录的特征。这些描述可能是显式的(例如, 一组规则定义), 也可能是隐式的(例如, 一个数学模型或公式)。

(4) 聚类分析。聚类分析是根据“物以类聚”的原理, 将本身没有类别的样本聚集成不同的组, 并且对每个这样的组进行描述的过程。其主要依据是聚集到同一个组中的样本应该彼此相似, 而属于不同组的样本应该足够不相似。聚类分析是分类分析的逆过程, 它的输入集是一组未标定的记录, 即输入的记录没有做任何处理, 目的是根据一定的规则, 合理地划分记录集合, 并用显式或隐式的方法描述不同的类别。

(5) 预测方法。预测方法与分类分析相似, 但预测是根据样本的已知特征估算某个连续类型的变量的取值的过程, 而分类分析则只是用于判别样本所属的离散类别而已。预测方法常用的技术是回归分析。

(6) 时间序列分析。时间序列分析是随时间而变化的事件序列, 目的是预测未来发展趋势, 或者寻找相似发展模式, 或者发现周期性的发展规律。

在实际应用中，以上分析方法有着不同的适用范围，经常被综合运用。

5.7.3 数据挖掘技术的应用

数据挖掘和数据仓库的协同工作，一方面可以迎合和简化数据挖掘过程中的重要步骤，提高数据挖掘的效率和能力，确保数据挖掘中数据来源的广泛性和完整性；另一方面，数据挖掘技术已经成为数据仓库应用中极为重要和相对独立的一个方面和工具。

数据挖掘和数据仓库的融合与互动发展，使数据挖掘得到了广泛的应用。目前，数据挖掘已经形成了多个分支，例如，空间数据挖掘（Spatial Data Mining, SDM）、多媒体数据挖掘（Multimedia Data Mining, MDM）和文本数据挖掘（Text Data Mining, TDM）等。

1. 空间数据挖掘

空间数据挖掘是指从空间数据库中抽取没有清楚表现出来的隐含的知识和空间关系，并发现其中有用的特征和模式的理论、方法和技术。SDM 是在数据挖掘的基础上，结合地理信息系统、遥感图像处理、全球定位系统、模式识别、可视化等相关的研究领域而形成的一个分支学科，也称为空间数据挖掘和知识发现（Spatial Data Mining and Knowledge Discovery, SDMKG）。空间数据与其他类型数据的本质区别是其空间属性。空间属性包括空间位置、距离、几何形状和大小等内容，并且可引申为空间个体之间的相互关系，例如，拓扑关系、方位关系和度量关系等，从而使得空间数据比其他类型的数据更为复杂，数据量异常巨大。空间数据的特点是其中隐含着更多、更为复杂的知识，因此，也使空间数据挖掘的研究更加困难和更具挑战性。

2. 多媒体数据挖掘

多媒体数据挖掘是在大量的多媒体数据中，通过综合分析视听特性和语义，发现隐含的、有效的、有价值的、可理解的模式，进而发现知识，得出事件的发展趋向和关联，为用户提供问题求解层次的决策支持能力。

MDM 相对于传统的数据挖掘有几个需要解决的问题。首先，多媒体数据为非结构化和异构数据。要在这些非结构化的数据上进行挖掘以获取知识，必须将它们转化为结构化数据，通过特征提取，用特征向量作为元数据建立元数据库，在此基础上进行数据挖掘。其次，多媒体数据的特征向量通常是数十维甚至数百维，如何对高维矢量进行数据挖掘也是需要考虑的重要问题。

3. 文本数据挖掘

文本数据挖掘是指从文本数据中抽取有价值的信息和知识的计算机处理技术。文本数据挖掘是应用驱动的，它在智能商务、信息检索、生物信息处理等方面都有广泛的应用。例如，客户关系管理、互联网搜索等。

按照挖掘的对象不同，可以将 TDM 分为基于单文档的数据挖掘和基于文档集的数据挖掘。基于单文档的数据挖掘对文档的分析并不涉及其他文档，其主要的挖掘技术有文本摘要和信息提取（包括名字提取、短语提取和关系提取等）；基于文档集的数据挖掘是对大规模的文档数据进行模式抽取，其主要的技术有文本分类、文本聚类、个性化文本过滤、文档作者归属和因素分析等。

TDM 可以分为三层：底层是 TDM 的基础领域，包括机器学习、数理统计和自然语言处理；

在此基础上是 TDM 的基本技术, 包括文本信息抽取、文本分类、文本聚类、文本数据压缩和文本数据处理; 最上层是两个主要应用领域, 分别是信息访问和知识发现, 信息访问包括信息检索、信息浏览、信息过滤和信息报告, 知识发现包括数据分析和数据预测。

5.8 非关系数据库

NoSQL 是非关系数据存储的广义定义, 它打破了长久以来关系数据库与 ACID 理论大一统的局面。NoSQL 数据存储不需要固定的表结构, 通常也不存在连接操作, 在大数据存取上具备关系数据库无法比拟的性能优势。该术语在 2009 年初得到了广泛认同。

5.8.1 概述

随着互联网 Web 2.0 网站的兴起, 非关系数据库成了一个极其热门的新领域, 非关系数据库产品的发展非常迅速。而传统的关系数据库在应对 Web 2.0 网站, 特别是超大规模和高并发的 SNS (Social Network Site, 社交网站) 类型的 Web 2.0 纯动态网站时已经显得力不从心, 暴露了很多难以克服的问题, 例如, 对数据库高并发读写的需求, 对海量数据的高效率存储和访问的需求, 对数据库的高可扩展性和高可用性的需求。

在基于 Web 的架构中, 数据库是最难进行横向扩展的, 当一个应用系统的用户量和访问量与日俱增的时候, 数据库却没有办法像 Web Server 和 App Server 那样简单地通过添加更多的硬件和服务节点来扩展性能和负载能力。对于很多需要提供 24 小时不间断服务的网站来说, 对数据库系统进行升级和扩展是非常痛苦的事情, 往往需要停机维护和数据迁移。

因此, 关系数据库在这些越来越多的应用场景下显得不那么合适了, 为了解决这类问题, 非关系数据库应运而生。

当今的应用体系结构需要数据存储能够在横向伸缩性上能够满足需求, 而 NoSQL 存储就是为了实现这个需求。Google 的 BigTable 与 Amazon 的 Dynamo 是非常成功的商业 NoSQL 实现。一些开源的 NoSQL 体系, 如 Facebook 的 Cassandra, Apache 的 HBase, 也得到了广泛认同。

NoSQL 数据库并没有统一的模型, 而且是非关系型的。常见的 NoSQL 数据库通过存储方式划分, 可分为文档存储、键值存储、列存储和图存储, 其具体分类和特点如表 5-13 所示。两种不同的 NoSQL 数据库之间的差异程度, 远远超过两种关系数据库之间的不同。一个优秀的 NoSQL 数据库必然适合某些场合或者某种应用。

表 5-13 NoSQL 数据库分类与特点

分 类	典型产品	应用场景	优 点	缺 点
文档存储	MongoDB、 CouchDB	Web 应用, 存储面向文档和半结构化数据	结构灵活, 可以根据 value 构建索引	缺乏统一的查询语法, 无事务处理能力
键值存储	Memcached、 Redis	内容缓存, 如会话、配置文件、参数等	扩展性好, 灵活性高, 大量操作时性能高	数据无结构化, 通常被当成字符串或者二进制数据, 通过键查询值

(续表)

分 类	典型产品	应用场景	优 点	缺 点
列存储	BigTable、HBase、Cassandra	分布式数据存储和管理	可扩展性强，查找速度快，复杂性低	功能局限，不支持事务的强一致性
图存储	Neo4j、OrientDB	社交网络、推荐系统，专注于构建系统图谱	支持复杂的图形算法	复杂性高，只能支持一定的数据规模

5.8.2 相关理论基础

1. CAP 理论

简单地说，CAP 理论就是对于一个分布式系统，一致性（Consistency）、可用性（Availability）和分区容忍性（Partition tolerance）三个特点最多只能三选二。

（1）一致性。一致性指系统在执行了某些操作后仍处在一个一致的状态，这点在分布式的系统中尤其明显。比如，某用户在一处对共享的数据进行了修改，那么所有有权使用这些数据的用户都可以看到这一改变。简言之，就是所有的结点在同一时刻有相同的数据。

（2）可用性。可用性指对数据的所有操作都应有成功的返回。高可用性则是在系统升级（软件或硬件）或在网络系统中的某些结点发生故障的时候，仍可以正常返回。简言之，就是任何请求不管成功或失败都有响应。

（3）分区容忍性。这一概念的前提是在网络发生故障的时候。在网络连接上，一些结点出现故障，使得原本连通的网络变成了一块一块的分区，若允许系统继续工作，那么就是分区可容忍的。

在数据库系统中，事务的 ACID 属性保证了数据库的一致性，比如银行系统中，转账就是一个事务，从原账户扣除金额，以及向目标账户添加金额，这两个数据库操作的总和构成一个完整的逻辑过程，具有原子的不可拆分特性，从而保证了整个系统中的总金额没有变化。然而，ACID 特性对于大型的分布式系统来说，可能并不总是适合，分布式系统更多的是强调可用性和分区容忍性。

由于 CAP 理论的存在，为了提高性能，出现了 ACID 的一种变种 BASE。BASE（Basic Availability, Soft-State, Eventual Consistency），即是一个弱一致性的理论，只要求最终一致性。

- Basic Availability: 基本可用。
- Soft State: 软状态，可以理解为“无连接”的，而与之相对应的 Hard State 就是“面向连接”的。
- Eventual Consistency: 最终一致性，最终整个系统（时间和系统的要求有关）看到的数据是一致的。

具体地说，如果选择了 CP（一致性和分区容忍性），那么就要考虑 ACID（传统关系数据库的基石，事务的 4 个特点）。如果选择了 AP（可用性和分区容忍性），那么就要考虑 BASE。如果选择了 CA（一致性和可用性），如 Google 的 BigTable，那么在网络发生分区的时候，将不能进行完整的操作。

ACID 和 BASE 的具体对比如表 5-14 所示。

表 5-14 ACID 和 BASE 的对比

ACID	BASE
强一致性	弱一致性
隔离性	优先考虑可用性
关注“操作提交”	尽力服务
允许事务嵌套	结果对精确度要求不高
可用性	更简单、更快速
比较“保守”	比较“激进”
难于演变	易于演变

2. 分区

现在,数据量的增加已经使得数据不可以仅在单一的计算机系统中存储(分布式的应用),尤其是为了保证数据的可靠性,有时需要复制备份。同时,为了一些规模性的操作(比如负载均衡)或者考虑到一些动态因素的影响(存储结点的改变),在设计中就要考虑“分区”的概念。

分区的一些主要方法如下:

(1) 内存缓存。缓存技术可以看成一种分区。内存中的数据库系统将使用频率最高的数据复制到缓存中,加快了将数据传递给用户的速度,同时也大大减轻了数据库服务器的负担。在分布式缓存中,缓存由很多带有分配好一定内存的进程组成,它们能够放置到不同的机器上,并且可以通过配置进行应用。它的协议可以在不同的编程语言中实现,同时在用户的应用中提供了简单的键值存储 API。它通过将键值哈希散列到缓存中来存储对象。

(2) 集群。数据库服务器集群在为用户提供服务时的透明性(用户感觉数据像是在同一个地方),是另外一个对数据进行分区的方法。然而,这种方法虽然能在某种程度上扩展系统数据持久层,可是集群本身的特性却仅仅只应用在了数据库管理系统的顶层,而并未在分布式最初的设计中得到应用。

(3) 读写分离。指定一台或多台主服务器,所有或部分的写操作被送至此,同时再设一定数量的副本服务器用以满足读请求。如果主服务器向至少一个用户异步复制数据,这是没有写延迟的,但如果主服务器在向最后一个用户写数据还没完成的时候就崩溃了,那么写操作将是无效的;如果主服务器向用户同步复制数据,这是有延迟的,这种更新不会丢失,但读请求却不能送达副本服务器。如果对一致性要求很高的话,无法避免进一步的写延迟。在这种情况下,如果主服务器崩溃了,那么有最新的数据的副本服务器将会成为新的主服务器。这种模型(主/从模型)在读写率很高的时候工作得很好。

(4) 范围分割技术/分片(Sharding)。该方法是对数据按照如下的方式进行分区操作:对数据的请求和更新在同一个结点上,并且对于分布在不同服务器上的数据存储和下载的量大致相同。从可靠性和负载均衡的观点看,数据的碎片也是需要被复制的,并且允许它们被写入主服务器的副本中和所有需要维护数据分区的副本服务器中。而为了做到这一点,需要在分区和存

储结点之间做一个映射。这个映射是动态还是静态取决于用户的应用、主服务器的“映射服务/组件”以及网络中用户应用于网络结点之间的基础结构。在分区场景中，关键在于如何将数据库中的对象映射到服务器上，通常采用哈希散列法。

3. 存储布局

存储布局确定了如何访问磁盘，以及如何直接影响性能，主要分为4种，分别是基于行的存储布局、列存储布局、带有局部性群组的列存储布局、LSM-Tree。

1) 行存储和列存储

行存储和列存储的主要区别在于，行存储将每条记录的所有字段的数据聚合存储，而列存储将所有记录中相同字段的数据聚合存储。行存储主要适用于OLTP，或者更新操作，尤其是插入、删除操作频繁的场合；而列存储主要适用于OLAP、数据仓库、数据挖掘等查询密集型应用。

列存储相对于行存储的优点主要有两个：

- 每个字段的数据聚集存储，在查询只需要少数几个字段的时候能大大减少读取的数据量。而查询密集型应用的特点之一就是查询一般只关心少数几个字段，而行存储每次必须读取整条记录。
- 既然一个字段的数据聚集存储，那就更容易为这种聚集存储设计更好的压缩/解压算法。

2) 带有局部性群组的列存储

带有局部性群组的列存储和列存储很相似，但是增加了局部性群组的特色。

局部性群组是在Google的关于BigTable的论文中第一次提到的。它是指根据需要将原来不存储在一起的数据，以列族为单位存储至单独的子表中。如用户对网站排名、语言等分析信息感兴趣，那么可以将这些列族放在单独的子表中，减少无用信息读取，改善存取效率。

3) LSM-Tree

LSM-Tree (Log Structured Merge Tree, 日志结构合并树) 与前面介绍的存储结构有所不同，前面介绍的存储结构在描述如何序列化逻辑数据结构，而LSM-Tree描述的则是为了满足高效、高性能、安全地读写的要求，如何有效地利用内存和磁盘存储。LSM-Tree的观点是由Patrick O'Neil在1996年率先提出的。LSM-Tree的算法思想主要用于解决日志记录索引的问题，这种应用的特点是数据量大、写速率高（2000条/秒），又要建立有效的索引来查找日志中的特定条目。Patrick O'Neil的做法是，在内存里维护一个相同的B树，当内存中的B树达到阈值时，批量进行滚动合并。LSM-Tree的典型例子就是Google的BigTable。

4. 查询模型

不同的NoSQL数据存储提供的查询功能有重大分歧。然而设计的键值存储仅提供通过主键或一些id字段查找的功能，并且缺乏查询更进一步域的能力，而其他数据存储（如文档数据库CouchDB和MongoDB）则允许某些复杂查询。在许多NoSQL数据库的设计中，相对于性能和可扩展性来说，丰富的动态查询功能已被省略的现状并不奇怪。另一方面，在使用NoSQL数据库时，有些使用情况下需要至少有一些非主键属性查询功能。

现在许多 NoSQL 数据库是基于 DHT (Distributed Hash Table, 分散哈希表) 模型的。为了访问和修改对象数据, 客户端要求提供对象的主键, 然后数据库再根据提供的主键进行相等匹配。

5.8.3 列数据库

传统的关系数据库都是以行为单位来进行数据存储的, 擅长进行以行为单位的数据处理, 比如特定条件数据的获取。因此, 关系数据库也被称为面向行的数据库。相反, 面向列的数据库是以列作为单位来进行数据存储的, 擅长进行以列为单位的数据处理。

面向列的数据库具有高扩展性, 即使数据增加也不会降低相应的处理速度, 所以它主要应用于需要处理大量数据的情况。另外, 利用面向列的数据库的优势, 也可以把它作为批处理程序的存储器, 来对大量数据进行更新。

列存储数据库的主要产品有 Google 的 BigTable、由 BigTable 衍生的 HyperTable 和 HBase、Cassandra 等。

5.8.4 文档数据库

在传统的数据库中, 数据被分割成离散的数据段, 而文档存储则是以文档为存储信息的基本单位。文档存储一般用类似 JSON 的格式存储, 存储的内容是文档型的。面向文档的数据库具有以下特征: 即使不定义表结构, 也可以像定义了表结构一样来使用; 面向文档的数据库可以通过复杂的查询条件来获取数据。虽然不具备事务处理和 JOIN 等关系数据库所具有的处理能力, 但其他的处理基本都可以实现。

在文档存储中, 文档可以很长、很复杂、无结构, 可以是任意结构的字段, 并且数据具有物理和逻辑上的独立性, 这就和具有高度结构化的表存储 (关系数据库的主要存储结构) 有很大的不同, 而最大的不同则在于它不提供对参数完整性和分布事物的支持。不过, 它们之间也并不排斥, 可以进行数据的交换。

MongoDB 是 10gen 公司开发的以高性能和可扩展性为特征的文档数据库, 也是 NoSQL 文档存储模式数据库中的重要一员。MongoDB 的最大特点就是无表结构。在保存数据和数据结构的时候, 会把数据和数据结构都完整地以 BSON 的形式保存起来 (BSON 是 JSON 的二进制编码格式, 但比 JSON 支持更加复杂的格式, 在空间利用上更加高效), 并把它作为值和特定的键进行关联。正是由于这种设计, 它不需要表结构, 因此它被称为文档数据库。

MongoDB 具有 4 个主要特征:

- (1) 高性能。提供 JSON、XML 等可嵌入数据快速处理功能; 提供文档的索引功能, 相对传统数据库而言, 大大提高查询速度。
- (2) 丰富的查询语言。为数据聚合、结构文档、地理空间提供丰富的查询功能。
- (3) 高可用性。提供数据冗余处理和故障自动转移的功能。
- (4) 水平扩展能力。通过集群将数据分布到多台机器, 而不是只提升单个节点的性能, 具体处理分为主从和权衡两种处理模式。

MangoDB 为 C、C#、.NET、Java、PHP 等各种开发语言提供了程序库。

5.8.5 键值数据库

键值存储模型是最简单也是最方便使用的数据模型，它支持简单的键对值的键值存储和提取。根据一个简单的字符串（键）能够返回一个任意类型的数据（值）。键值存储最大的好处是不用为值指定一个特定的数据类型，这样就能在值里存储任意类型的数据。系统将这些信息按照 BLOB 大对象进行存储，当收到检索请求时，返回同样的 BLOB。由应用来决定被使用的数据是什么类型，如字符串、图片和 XML 文件等。键值存储数据库的主要特点是具有极高的并发读写性能。

键值存储中的键是很灵活的，可以是图片名称、网页 URL 或者文件路径名，它们指向那些图片、网页和对应的文档。

键值存储中存在三种操作：put、get 和 delete。这三种操作规定了程序员与键值存储交互的基本方式。应用开发者使用 put、get 和 delete 函数访问和操作键值存储：

- put(\$key as xs:string, \$value as item()) 对表添加一个新的键值对，并且当键存在时，更新键对应的值。
- get(\$key as xs:string) as item() 根据给出的任意键返回键对应的值，如果键值存储中没有该键，将返回一个错误信息。
- delete(\$key as xs:string) 将键和对应的值从表中删除，如果键值存储中没有该键，将返回一个错误信息。

根据数据的保存方式，键值存储的数据库可以分为三种，分别是临时性保存、永久性保存和两者兼具型。

1) 临时性保存类型

临时性的保存有可能丢失数据。这类数据库一般把数据都保存在内存中，这样读和写的速度都非常快，但是当数据库停止或者机器重启后，数据就不存在了。

2) 永久性保存类型

永久性的保存不会丢失数据。这类数据库一般不把数据保存在内存中，而是把数据保存在硬盘上。与临时性保存在内存中读和写数据比较起来，由于永久性保存牵扯到对硬盘的 I/O 操作，所以性能上的差距是显而易见的，但是它最大的优势是不会丢失数据。

3) 两者兼具型

这类数据库兼具临时性保存和永久性保存的优点。比如典型的 Redis 系统，它首先把数据保存在内存中，平时都在内存中进行读和写操作。在满足特定条件（默认是 15 min 一次以上，5 min 10 个以上或 1 min 1000 个以上的键发生变化）时，将数据写入硬盘上保存。这样既确保了内存中数据处理的高速性，又通过写入硬盘来保证数据的永久性。这种类型的数据库特别适合处理数组类型的数据。

键值存储产品主要有亚马逊的 Memcached、Redis、Dynamo、Project Voldemort、Tokyo Tyrant、Riak、Scalaries 这几个数据库，这里主要介绍一下 Redis。

Redis 是一种主要基于内存存储和运行，能够快速响应的键值数据库。属于临时性保存和永久性保存兼具的类型，有点像 Memcached，整个数据库统统加载在内存当中进行操作，但是通

过定期异步操作把数据库数据 flush 到硬盘上进行保存。因为是纯内存操作，Redis 的性能非常出色，每秒可以处理超过 10 万次读写操作。

Redis 的数据库存储模式，是在键值（Key-Value）基本存储原理的基础上进行细化分类，构建了具有自身特点的数据结构类型。像 MySQL 这样的关系数据库，表的结构比较复杂，会包含很多字段，可以通过 SQL 语句来实现非常复杂的查询需求。而 Redis 客户只包含“键”和“值”两部分，只能通过“键”来查询“值”。正是因为这样简单的存储结构，让 Redis 的读写效率非常高。键的数据类型是字符串，但是为了丰富数据存储的方式，方便开发者使用，值的数据类型很多，它们分别是字符串、列表、字典、集合、有序集合。

5.8.6 图数据库

图存储在那些需要分析对象之间的关系或者通过一个特定的方式访问图中所有节点的应用中尤为重要。图存储针对有效存储图节点和联系进行了优化，让你可以对这些图结构进行查询。图数据库对于那些对象之间具有复杂关系的业务问题很有用，如社交网络、规则引擎、生成组合和那些需要快速分析复杂网络结果并从中找出模式的图系统。

图存储是一个包含一连串的节点和关系的系统，当它们结合在一起时，就构成了一个图。图存储有三个字段：节点、关系和属性。图节点通常是现实世界中对象的表现，如人名、组织、电话号码、网页或计算机节点。而关系可以被认为是这些对象之间的联系，通常被表示为图中两个节点之间连接线。

如果图存储的节点众多、关系复杂、属性很多，那么传统的关系数据库将要建很多大型的表，并且表的很多列可能是空的，在查询时还极有可能进行多重 SQL 语句的嵌套。可是图存储就可以很优异，基于图的很多高效的算法可以大大提高执行效率。